Course – TFC

Course Instructor

Dr. Umadevi V

Department of CSE, BMSCE

Webpage:https://sites.google.com/site/drvumadevi/

Unit-5

Problems That Computers Cannot Solve

The Turing Machine, Programming Techniques for Turing Machines, Extensions to the Basic Turing Machine, Restricted Turing Machines, Turing Machines and Computers, Definition of Post's Correspondence Problem, A Language That Is Not Recursively Enumerable, An Undecidable Problem That is RE

Other Undecidable Problems

Problems That Computers Cannot Solve

- Specific Problems we cannot solve using a computer are called “Undecidable”

Example

Program to print "Hello World"

```
main()
{
    printf("hello, world\n");
}
```

Fermat's last theorem expressed as a hello world program

```
int exp(int i, n)
/* computes i to the power n */
{
    int ans, j;
    ans = 1;
    for (j=1; j<=n; j++) ans *= i;
    return(ans);
}

main ()
{
    int n, total, x, y, z;
    scanf("%d", &n);
    total = 3;
    while (1) {
        for (x=1; x<=total-2; x++)
            for (y=1; y<=total-x-1; y++) {
                z = total - x - y;
                if (exp(x,n) + exp(y,n) == exp(z,n))
                    printf("hello, world\n");
            }
        total++;
    }
}
```

Hello World Problem: Determine whether a given C program, with given input, prints "Hello World"

Note: In number theory, **Fermat's Last Theorem** states that no three positive integers a, b, and c satisfy the equation $a^n + b^n = c^n$ for any integer value of n greater than 2.

Example

The program (Fermat) takes an input n and looks for positive integer solutions to equation

$$x^n + y^n = z^n$$

If the program finds a solution, it prints `hello, world`

If it never finds integer x, y, z to satisfy the equation, then it continues searching **forever**, and never prints `hello, world`

If the value of n is 2, then it will find combinations of integers and thus:

For input $n = 2$ the program prints `hello, world`

For any integer $n > 2$, the program will never find a triple of positive integers to satisfy $x^n + y^n = z^n$

```
int exp(int i, n)
/* computes i to the power n */
{
    int ans, j;
    ans = 1;
    for (j=1; j<=n; j++) ans *= i;
    return(ans);
}

main ()
{
    int n, total, x, y, z;
    scanf("%d", &n);
    total = 3;
    while (1) {
        for (x=1; x<=total-2; x++)
            for (y=1; y<=total-x-1; y++) {
                z = total - x - y;
                if (exp(x,n) + exp(y,n) == exp(z,n))
                    printf("hello, world\n");
            }
        total++;
    }
}
```

Our hello-world problem

- Is there a program H that could examine any program P and any input I for P, and tell whether P, run with I as its input, would print *hello, world*?
- The answer is: undecidable!
  - That is, there exists no such program H.
  - We can prove this by contradiction next.

Hypothetical “Hello, World” Tester

We want to prove that no program **H**, called hypothetical “Hello, World” tester, as mentioned previously exists by contradiction using the following steps.

Step 1: assume H exists in a form as shown in Fig. 8.1

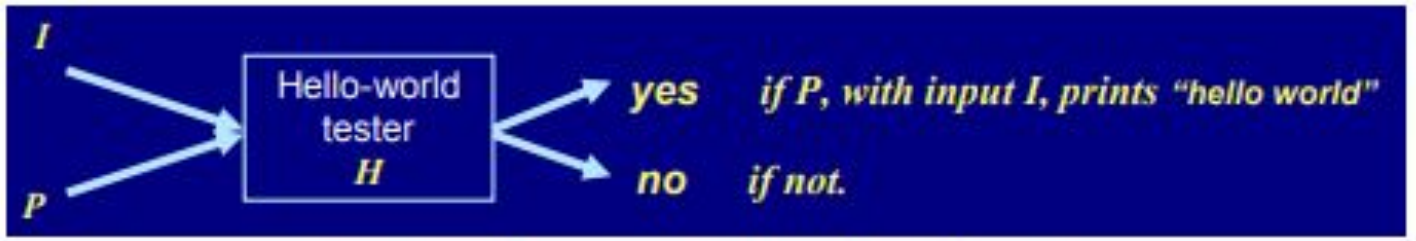


Fig. 8.1 A hypothetical “Hello, World” tester.

Step 2: Transform H into another form H2 in a simple way which can be done by C programs.

Step 3: prove that H2 does not exist and so that H does not exist, either

Hypothetical "Hello, World" Tester (Contd…)

Implementation of Step 2 in previous slide

(1) Transform H to H1 in a way as illustrated by Fig. 8.2

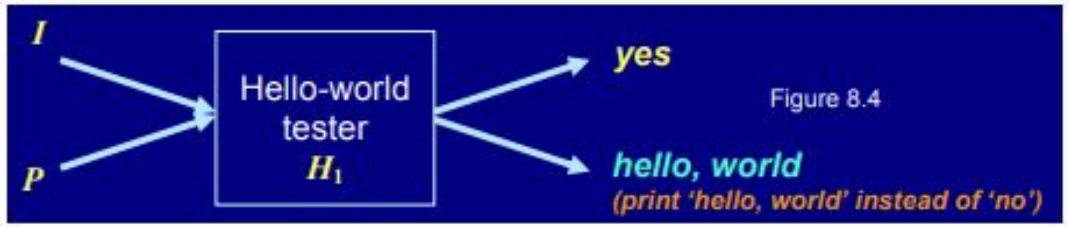


Fig. 8.2 A transformed "hello-world tester" H_1.

(2) Transform H1 to H2 in a way as illustrated by Fig. 8.3

Fig. 8.2 A second transformed "hello-world tester" H_2.

The function of H2 constructed in **Step 2** is given any program P as input,

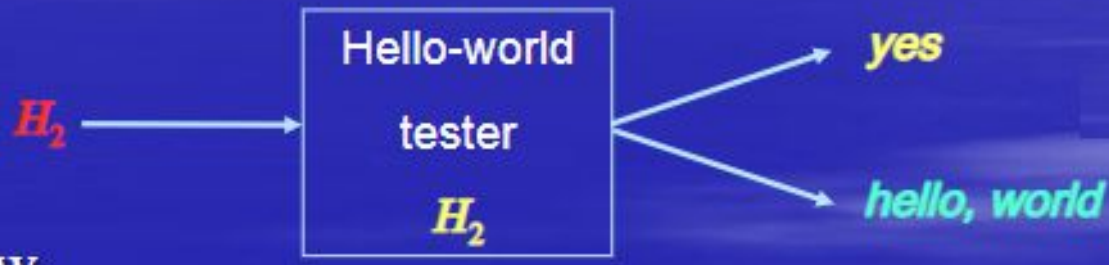
if P prints ***hello, world*** as first output, then H2 makes output **yes**;

if P **does not prints hello, world** as first output, then H2 prints **hello, world**

Hypothetical "Hello, World" Tester (Contd…)

Implementation of **Step 3** above (proving H2 does not exist) ---

- Prove H_2 does not exist by contradiction as follows (cont'd) –



- Now,

(1) if the box H_2, given itself as input, makes output *yes*, then it means that

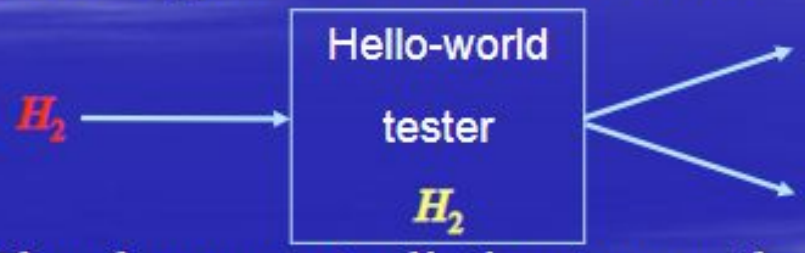
the input H_2, given itself as input, prints hello world as first output.

But this is contradictory because we just suppose that H_2, given itself as input, makes output *yes*.

Hypothetical "Hello, World" Tester (Contd…)

Implementation of **Step 3** above (proving H2 does not exist) ---

- Prove H_2 does not exist as follows (cont'd) –



- The above contradiction means the other alternative must be true (since it must be one or the other), that is ---

(2) the box H_2, given itself as input, prints ***hello, world*** . This then means that

such H_2, when taken as input H_2 to the box H_2(itself), will make the box H_2 to make output ***yes***.

Contradiction again because we just say that the box H_2, given itself as input, prints ***hello, world*** .

Reducing One Problem to Another

- Now we have an undecidable problem, which can be used to prove other undecidable problems by a technique of *problem reduction*.
  - That is, if we know P_1 is undecidable, then we may *reduce* P_1 *to a new problem* P_2, so that we can prove P_2 undecidable by contradiction in the following way
    - *If* P_2 *is decidable, then* P_1 *is decidable.*
    - *But* P_1 *is known undecidable. So, contradiction!*
    - *Consequently,* P_2 *is undecidable.*
- An illustration of the above idea is illustrated in Fig. 8.4.

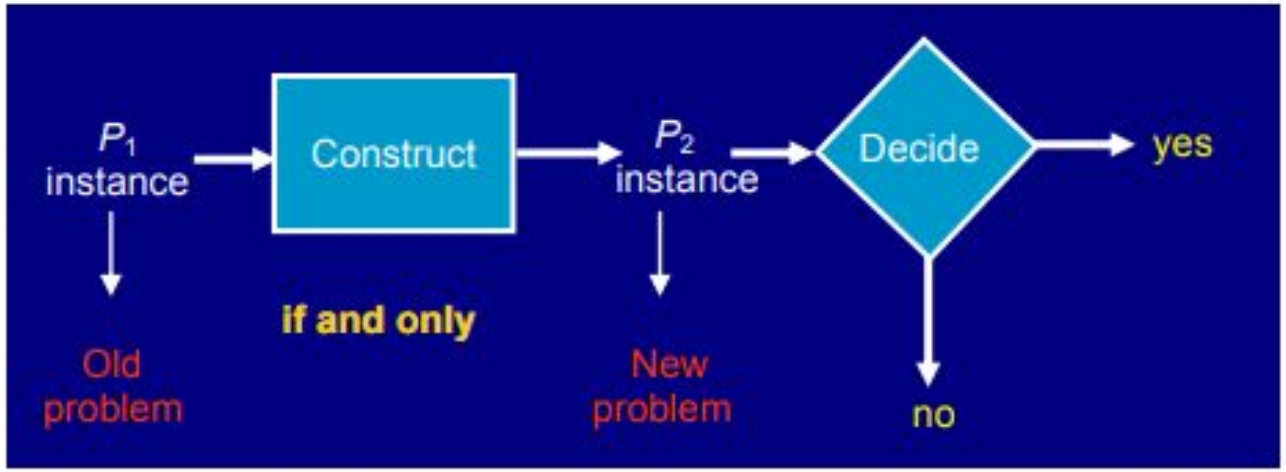


Fig. 8.4 An illustration of reducing one problem to another.

Propose of the theory of undecidable problems

To provide guidance to programmers about what they might or might not be able to accomplish through programming

Pragmatic impact:

- **Intractable problems:**
  - Decidable problems.
  - Require large amount of time to solve them.
  - Contrary to undecidable problems, which are usually rarely attempted in practice, the intractable problems are *faced everyday*.
  - Yield to small modifications in the requirements or to heuristc solutions.

- We need tools that allow us to prove everyday questions undecidable or intractable
- We need to rebuild our theory of undecidability, based not on C programs, but based on a very simple model of computer called **Turing Machine**

The Turing Machine ™

Informally...

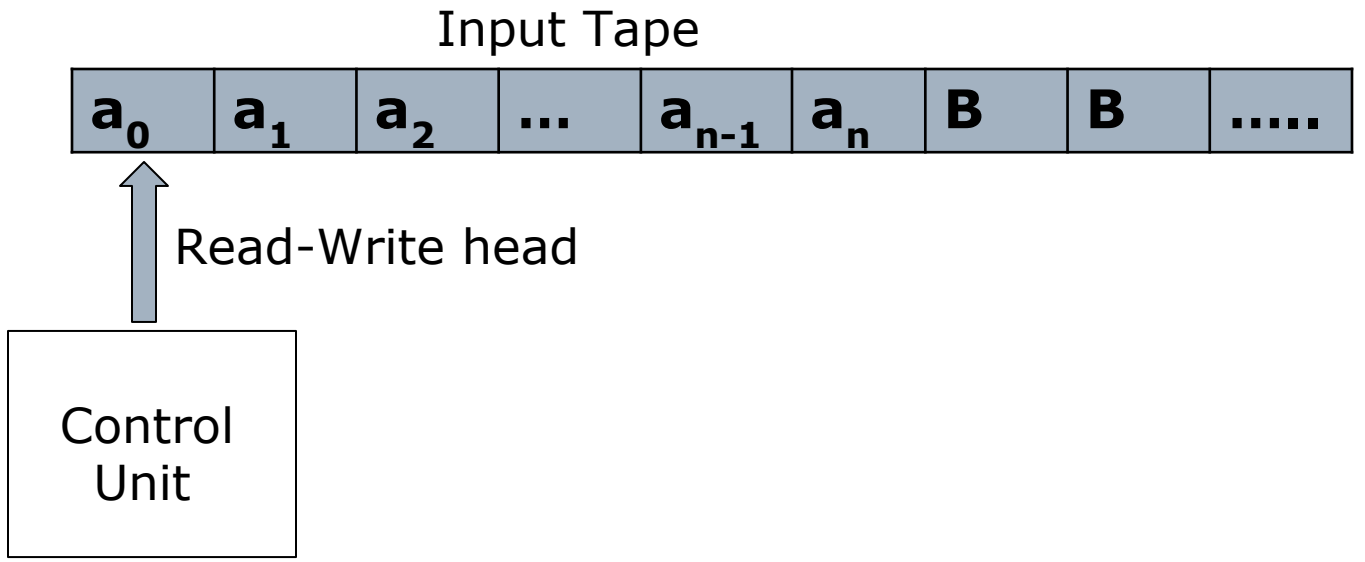
- TM is essentially a **finite automaton** that has a single **tape of infinite length** on which it may read and write data.
- Advantage of TM over programs as representation of what can be computed: TM is sufficiently simple that we can represent its configuration precisely, using a simple notation much like ID's of a PDA.

History

- 1936: Alan Turing proposed the TM as a model of *any possible computation*.
- This model is computer-like, rather than program-like, even though true electronic or electromechanical computers were several years in the future.
- All the serious proposals for a model of computation have the same power; *i.e.*, they compute the same functions and recognize the same languages.

Turing Machine (TM)

Turing machine model



Formal Definition of Turning Machine

P=(Q, Σ , Γ , δ , q_0, B, F)

- Q is set of **states**
- ∑ is set of **input alphabet**
- Γ is set of **tape symbols**
- q_0 ∈ Q , is the start state
- B is special symbol indicating blank character
- F is the set of final states
- δ is a transition function which maps QxΓ to QxΓ x{L, R}

Note: L and R indicate Left or Right direction move

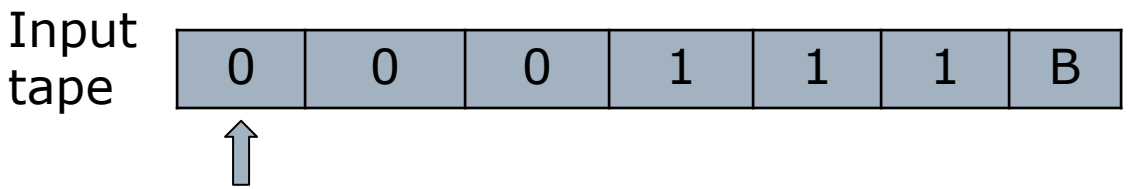
Question

Design Turing Machine for L= $\{0^n1^n, n>=1\}$

Example:

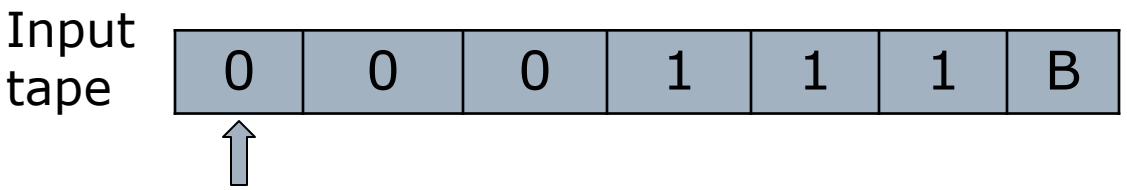
Valid String: 000111

Invalid String:00111



Question

Design Turing Machine for L= $\{0^n1^n, n>=1\}$
Example:
Valid String: 000111
Invalid String:00111



- **Approach used –**
  First replace a 0 from front by X, then keep moving right till you find a 1 and replace this 1 by Y.

  Now keep moving left till you find a X. When you find it, move a right, then follow the same procedure as above.
- A condition comes when you find a X immediately followed by a Y. At this point we keep moving right and keep on checking that all 1's have been converted to Y. If not then string is not accepted. If we reach B then string is accepted.

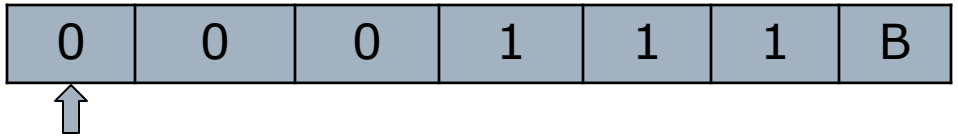
Rough Slide to Explain "Designing Turing Machine for L= $\{0^n1^n, n>=1\}$"

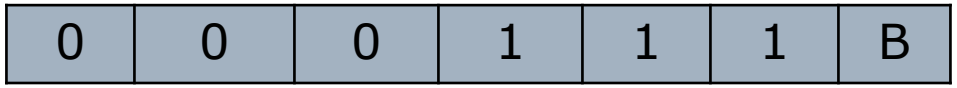
Approach used –

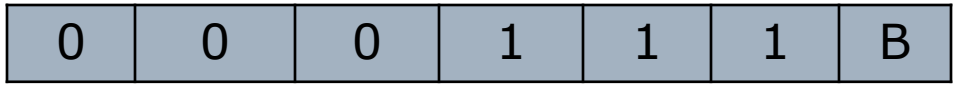
First replace a 0 from front by X, then keep moving right till you find a 1 and replace this 1 by Y.

Now keep moving left till you find a X. When you find it, move a right, then follow the same procedure as above.

A condition comes when you find a X immediately followed by a Y. At this point we keep moving right and keep on checking that all 1's have been converted to Y. If not then string is not accepted. If we reach B then string is accepted.







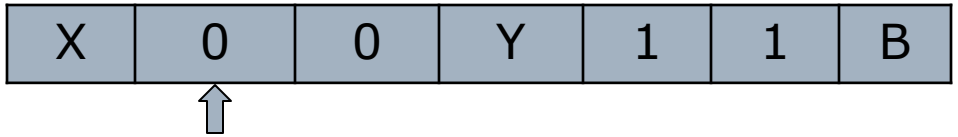
Rough Slide to Explain "Designing Turing Machine for L= $\{0^n1^n, n>=1\}$"

Approach used –

First replace a 0 from front by X, then keep moving right till you find a 1 and replace this 1 by Y.

Now keep moving left till you find a X. When you find it, move a right, then follow the same procedure as above.

A condition comes when you find a X immediately followed by a Y. At this point we keep moving right and keep on checking that all 1's have been converted to Y. If not then string is not accepted. If we reach B then string is accepted.



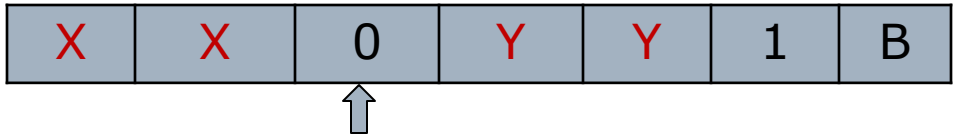
Rough Slide to Explain "Design Turing Machine for L= $\{0^n1^n, n>=1\}$"

Approach used –

First replace a 0 from front by X, then keep moving right till you find a 1 and replace this 1 by Y.

Now keep moving left till you find a X. When you find it, move a right, then follow the same procedure as above.

A condition comes when you find a X immediately followed by a Y. At this point we keep moving right and keep on checking that all 1's have been converted to Y. If not then string is not accepted. If we reach B then string is accepted.



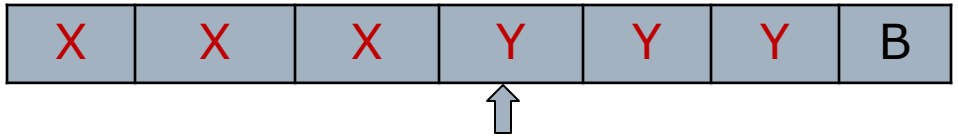
Rough Slide to Explain "Designing Turing Machine for L= $\{0^n1^n, n>=1\}$"

Approach used –

First replace a 0 from front by X, then keep moving right till you find a 1 and replace this 1 by Y.

Now keep moving left till you find a X. When you find it, move a right, then follow the same procedure as above.

A condition comes when you find a X immediately followed by a Y. At this point we keep moving right and keep on checking that all 1's have been converted to Y. If not then string is not accepted. If we reach B then string is accepted.



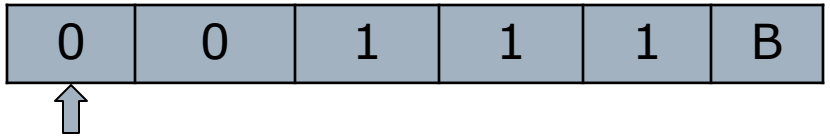
Rough Slide to Explain "Designing Turing Machine for L= $\{0^n1^n, n>=1\}$"

Approach used –

First replace a 0 from front by X, then keep moving right till you find a 1 and replace this 1 by Y.

Now keep moving left till you find a X. When you find it, move a right, then follow the same procedure as above.

A condition comes when you find a X immediately followed by a Y. At this point we keep moving right and keep on checking that all 1's have been converted to Y. If not then string is not accepted. If we reach B then string is accepted.



Invalid String

Rough Slide to Explain "Designing Turing Machine for L= $\{0^n1^n, n>=1\}$"

Approach used –

First replace a 0 from front by X, then keep moving right till you find a 1 and replace this 1 by Y.

Now keep moving left till you find a X. When you find it, move a right, then follow the same procedure as above.

A condition comes when you find a X immediately followed by a Y. At this point we keep moving right and keep on checking that all 1's have been converted to Y. If not then string is not accepted. If we reach B then string is accepted.

Invalid String

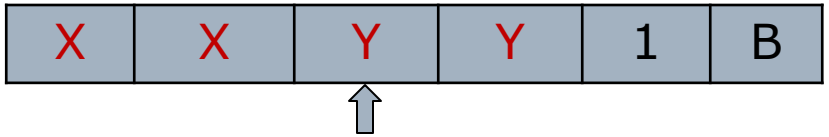
Rough to Explain "Designing Turing Machine for L= $\{0^n1^n, n>=1\}$"

Approach used –

First replace a 0 from front by X, then keep moving right till you find a 1 and replace this 1 by Y.

Now keep moving left till you find a X. When you find it, move a right, then follow the same procedure as above.

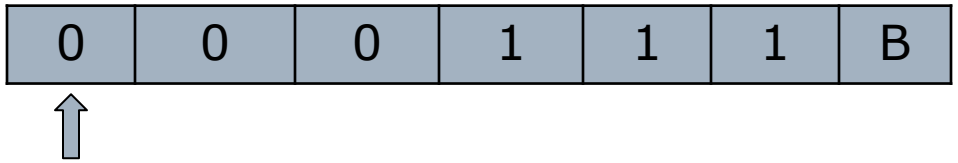
A condition comes when you find a X immediately followed by a Y. At this point we keep moving right and keep on checking that all 1's have been converted to Y. If not then string is not accepted. If we reach B then string is accepted.



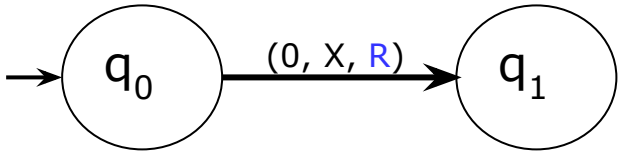
Invalid String

Design Turing Machine for L= $\{0^n1^n, n>=1\}$

Logic



Step-1: Replace 0 by X and move right, Go to state q1.

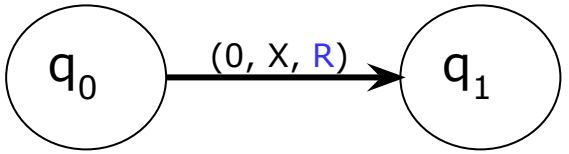


Design Turing Machine for L= $\{0^n1^n, n>=1\}$

Logic

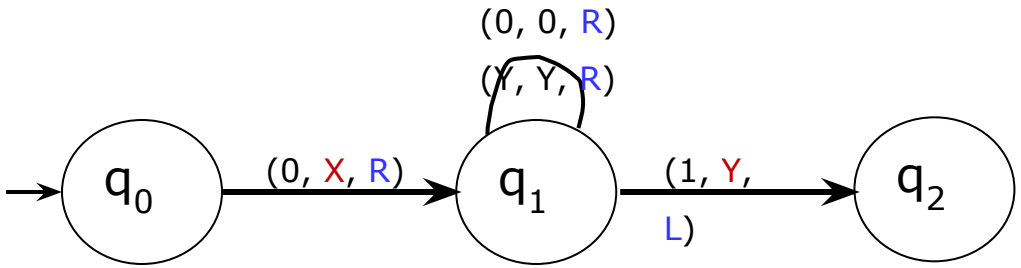
0	0	0	1	1	1	B

Step-1: Replace 0 by X and move right, Go to state q1.



X	0	0	Y	1	1	B

Step-2: Replace 0 by 0 and move right, Remain on same state.
Replace Y by Y and move right, Remain on same state.
Replace 1 by Y and move left, go to state q2.

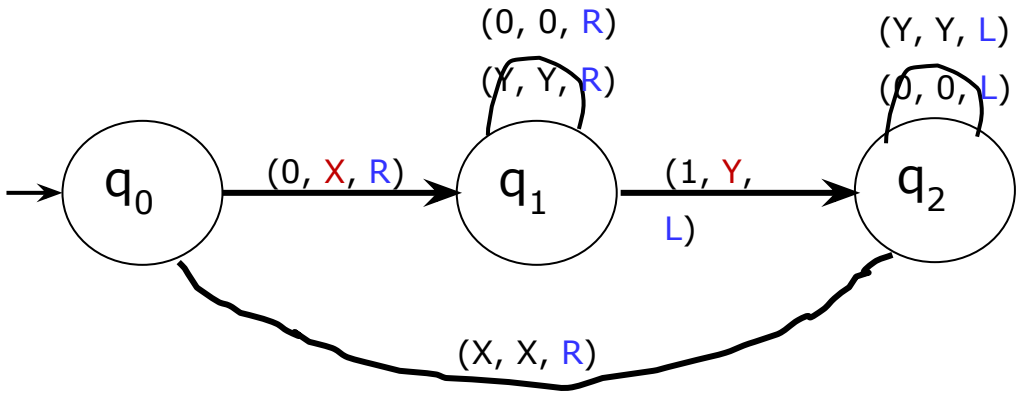


Design Turing Machine for L= $\{0^n1^n, n>=1\}$

Logic

X	0	0	Y	1	1	B

Step-3:
Replace 0 by 0 and move left, Remain on same state
Replace Y by Y and move left, Remain on same state
Replace X by X and move right, go to state q0.

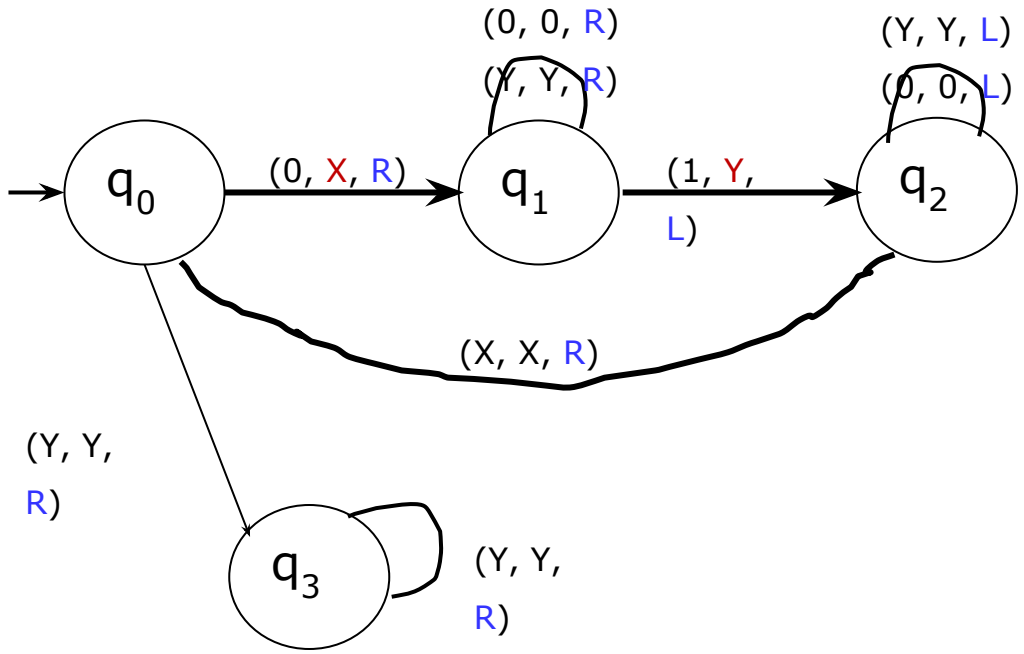


Design Turing Machine for L= {0^n1^n, n>=1}

Logic

X	0	0	Y	1	1	B

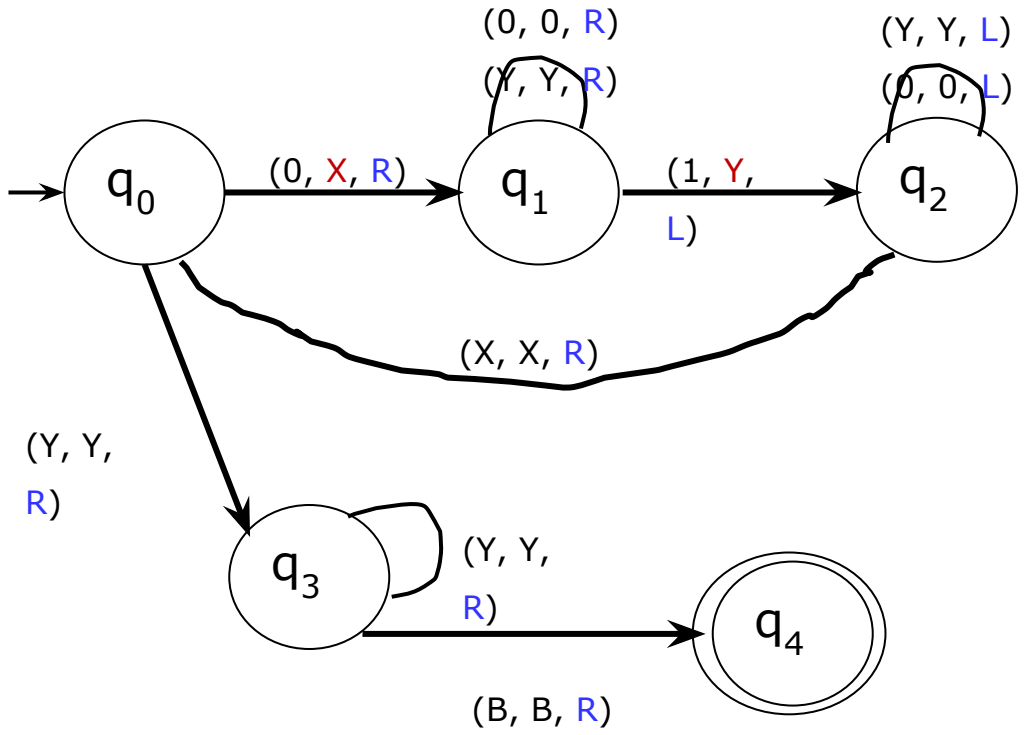
Step-4:
If symbol is Y replace it by Y and move right and Go to state q3
Else go to step 1



Design Turing Machine for L= $\{0^n1^n, n>=1\}$

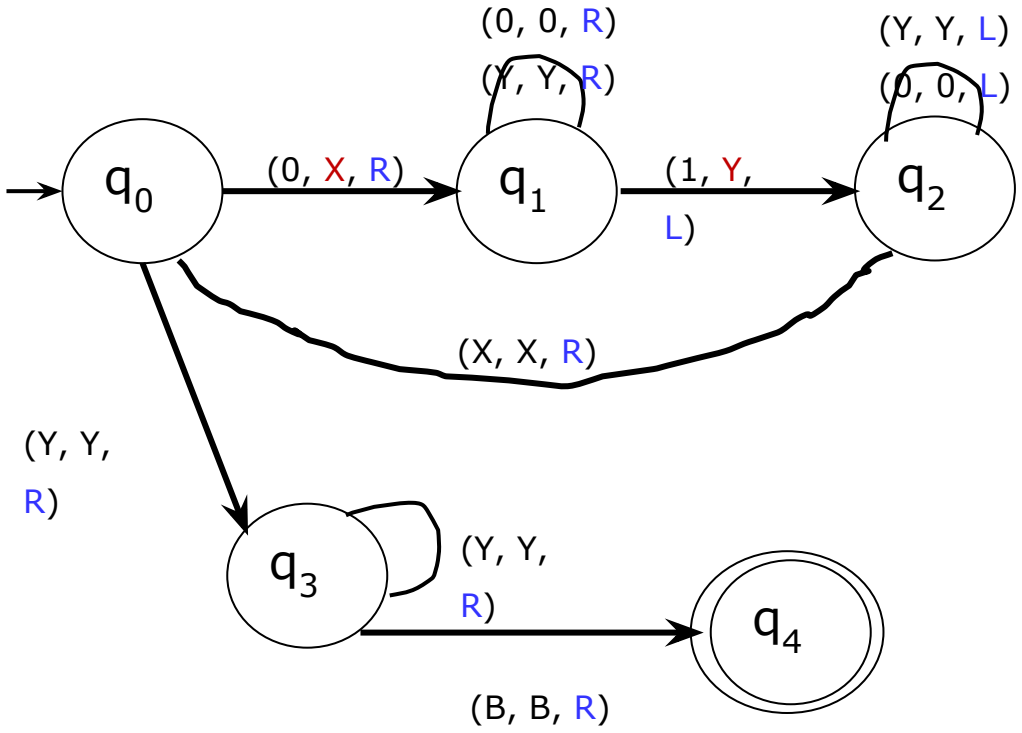
Logic

Step-5:
Replace Y by Y and move right, remain on same state
If symbol is B replace it by B and move right
STRING IS ACCEPTED, GO TO FINAL STATE q4



Design Turing Machine for L= $\{0^n1^n, n>=1\}$

Transition diagram



Design Turing Machine for L= $\{0^n1^n, n>=1\}$

Formal Definition

M=(Q, Σ , Γ , δ , q_0, B, F)

Q = $\{q_0, q_1, q_2, q_3, q_4\}$

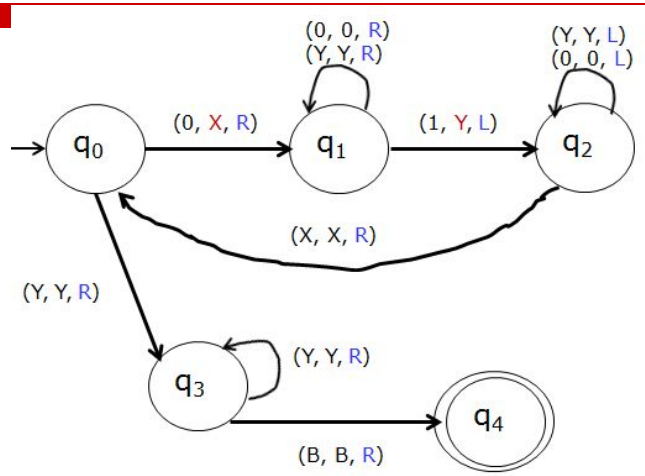
Σ = {0, 1}

Γ = {0, 1, X, Y, B}

q_0 = q_0

B = B

F = $\{q_4\}$



Transition Diagram

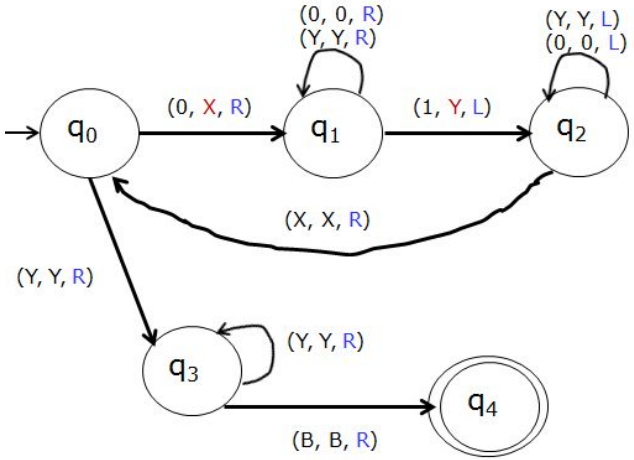
δ Transition table

State	Input tape Symbols				
	0	1	X	Y	B
q_0	(q_1, X , R)	---	---	(q_3, Y , R)	--
q_1	(q_1, 0 , R)	(q_2, Y, L)	---	(q_1, Y , R)	---
q_2	(q_2, 0 , L)	---	(q_0, X,R)	(q_2, Y,L)	---
q_3	---	---	---	(q_3, Y , R)	(q_4,B,R)
q_4	---	---	---	---	---

TM for L= $\{0^n1^n, n>=1\}$ (Contd….)
Rough Slide

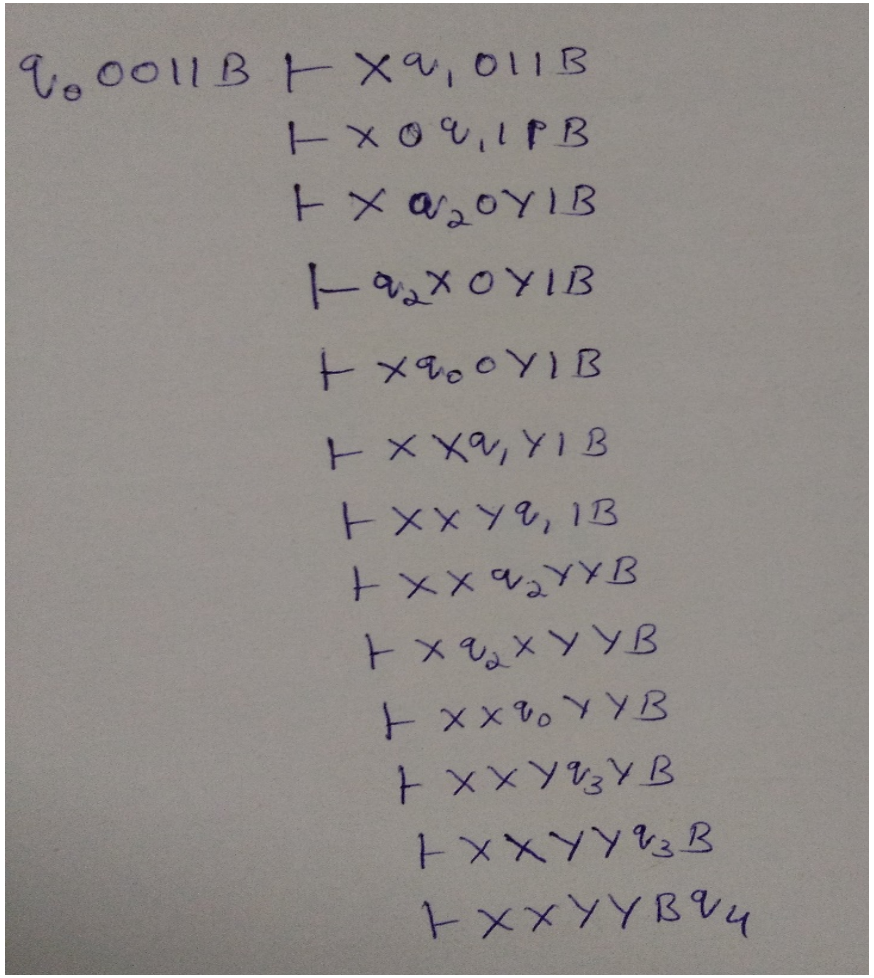
Instantaneous description for the string 0011

$q_0$0011B



TM for L= $\{0^n1^n, n>=1\}$ (Contd….)

Instantaneous description for the string 0011



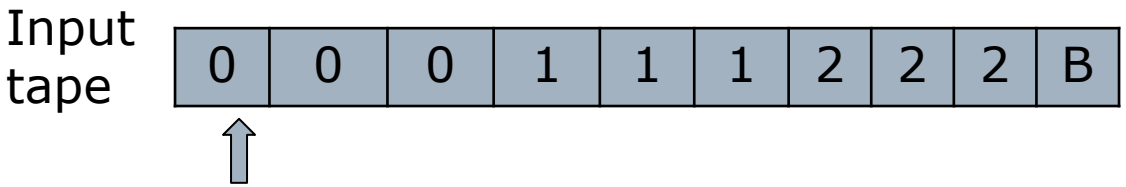
Question

Design Turing Machine for L= $\{0^n1^n2^n, n>=1\}$

Example:

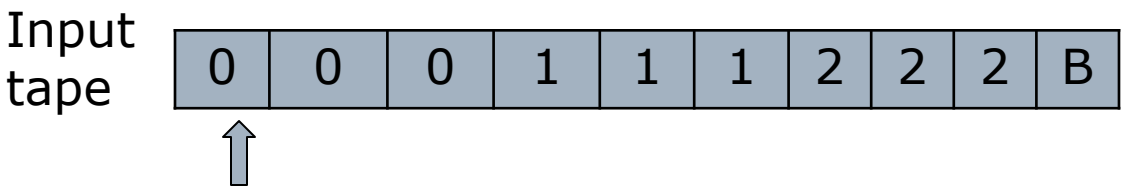
Valid String: 000111222

Invalid String:00111222



Question

Design Turing Machine for L= $\{0^n1^n2^n, n>=1\}$
Example:
Valid String: 000111222
Invalid String:00111222



Assumption: We will replace 0 by X, 1 by Y and 2 by Z
Approach used –
- First replace a 0 from front by X, then keep moving right till you find a 1 and replace this 1 by Y. Again, keep moving right till you find a 2, replace it by Z and move left. Now keep moving left till you find a X. When you find it, move a right, then follow the same procedure as above.

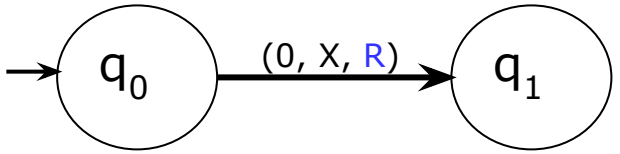
- A condition comes when you find a X immediately followed by a Y. At this point we keep moving right and keep on checking that all 1's and 2's have been converted to Y and Z. If not then string is not accepted. If we reach **B** and final state then string is accepted.

Design Turing Machine for L= {$0^n1^n2^n$, n>=1}

Logic

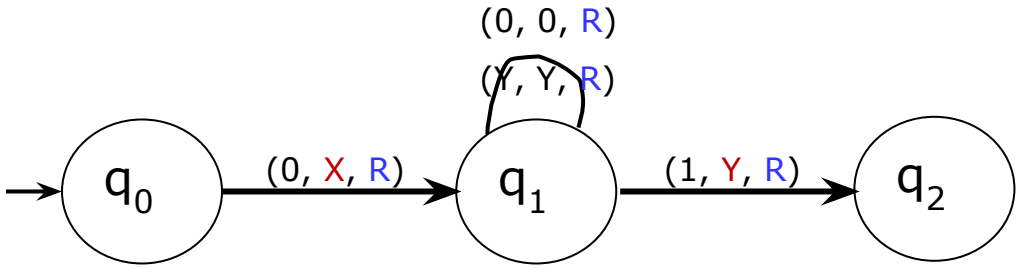
0	0	1	1	2	2	B

Step-1: Replace 0 by X and move right, Go to state q1.



X	0	Y	1	2	2	B

Step-2: Replace 0 by 0 and move right, Remain on same state
Replace Y by Y and move right, Remain on same state
Replace 1 by Y and move right, go to state q2.

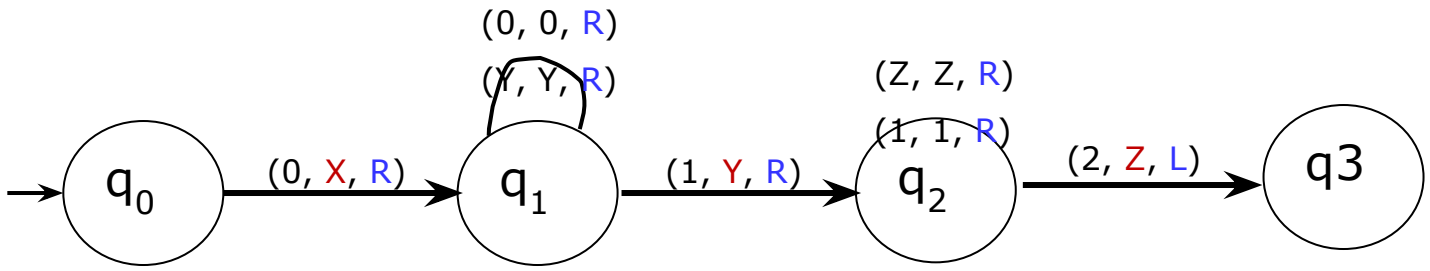


Design Turing Machine for L= {$0^n1^n2^n$, n>=1}

Logic

X	0	Y	1	2	2	B

Step-3:
Replace 1 by 1 and move right, Remain on same state
Replace Z by Z and move right, Remain on same state
Replace 2 by Z and move left, go to state q3.

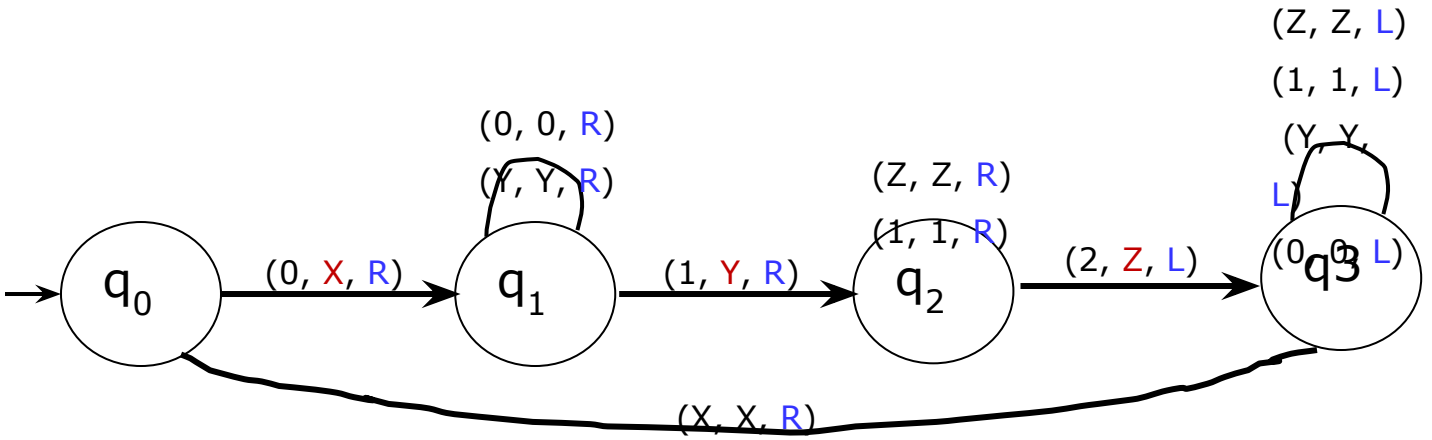


Design Turing Machine for L= {$0^n1^n2^n$, n>=1}

Logic

X	0	Y	1	Z	2	B

Step-4:
Replace Z by Z and move left, Remain on same state
Replace 1 by 1 and move left, Remain on same state
Replace Y by Y and move left, Remain on same state
Replace 0 by 0 and move left, Remain on same state
Replace X by X and move right, go to state q0.

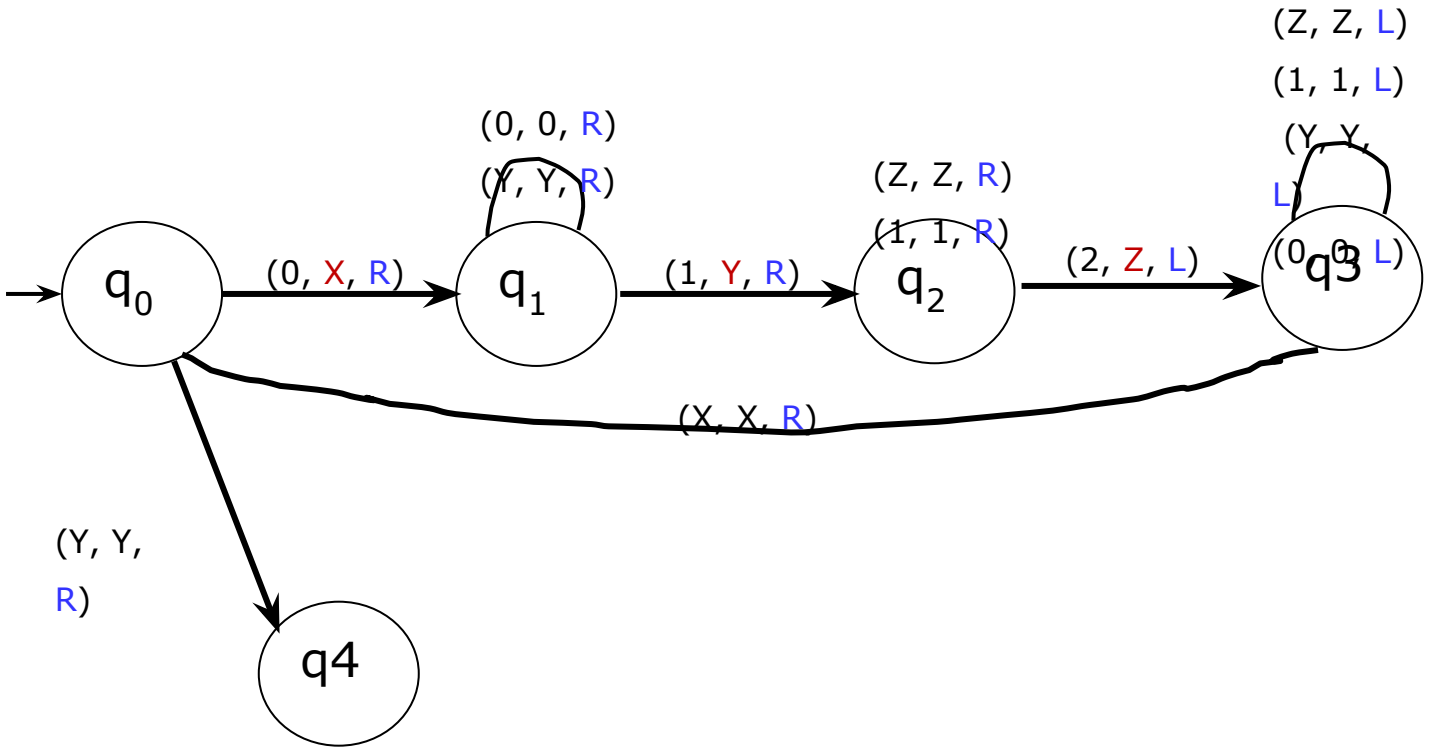


Design Turing Machine for L= {$0^n1^n2^n$, n>=1}

Logic

X	X	Y	Y	Z	Z	B

Step-5:
If symbol is Y replace it by Y and move right and Go to state q4
Else go to **Step-1**

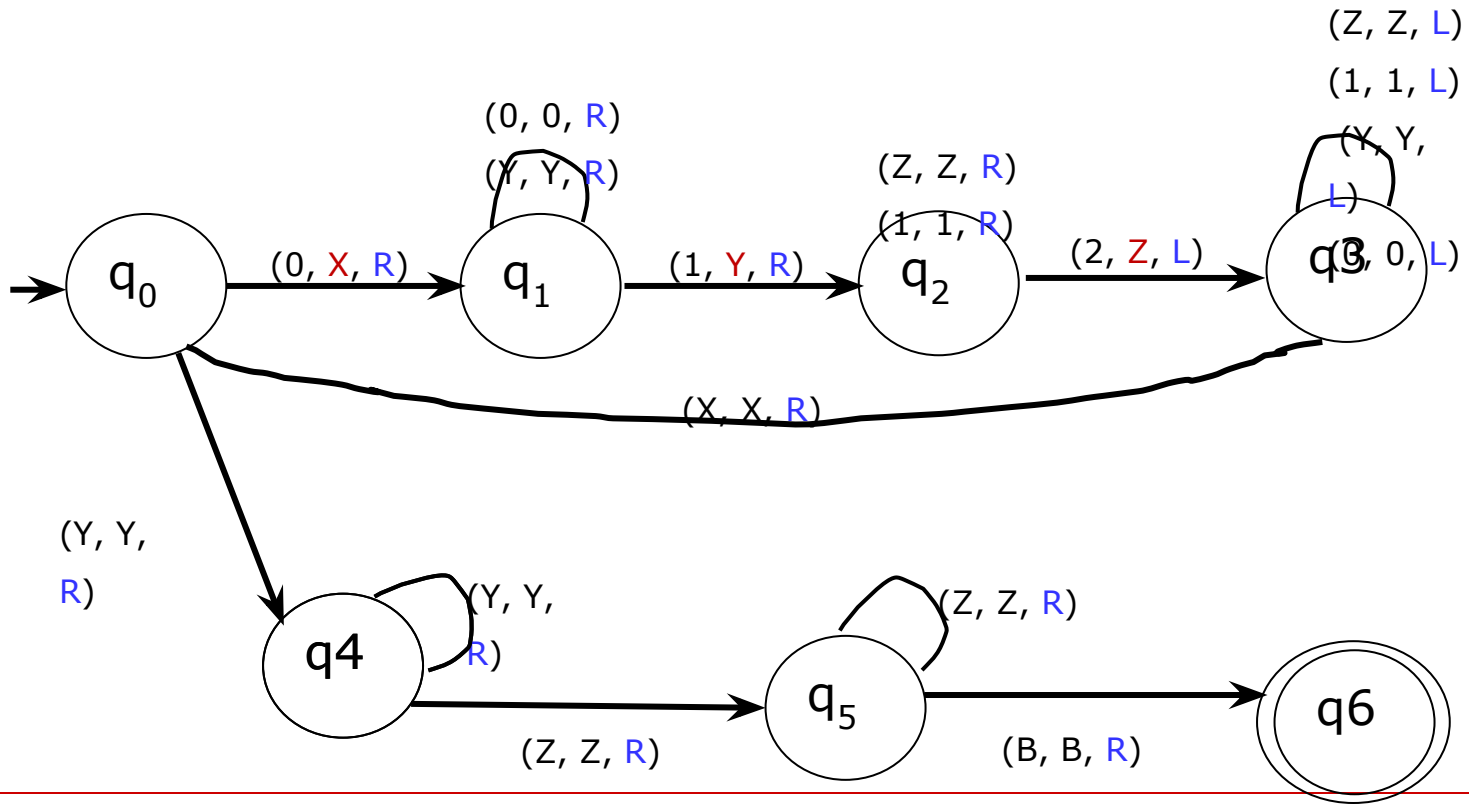


Design Turing Machine for L= {$0^n1^n2^n$, n>=1}

Logic

X	X	Y	Y	Z	Z	B

Step-6:
Replace Y by Y and move right, Remain on same state
If the symbol is Z, replace it by Z and move right and go to the state q5
Step-7:
Replace Z by Z and move right, Remain on same state
If symbol is B replace it by B and move right, STRING IS ACCEPTED, GO TO FINAL STATE q6



Complete design of Turing Machine for L= $\{0^n1^n2^n, n>=1\}$

Formal Definition

M=(Q, Σ , Γ , δ , q_0, B, F)

Q = $\{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$

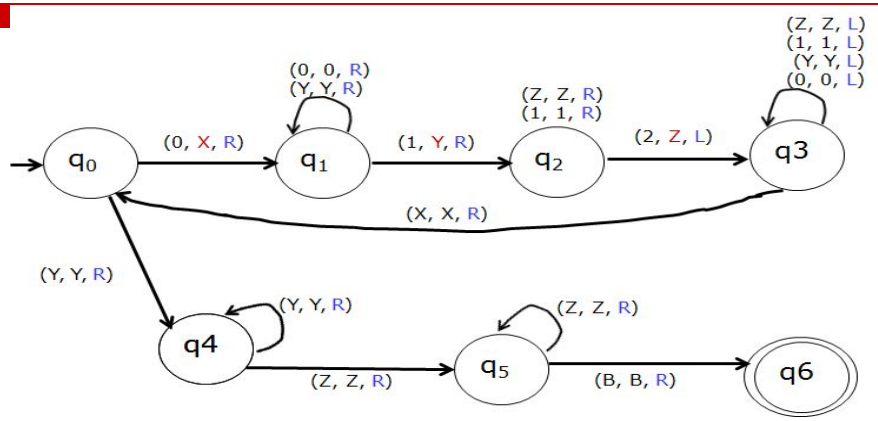
Σ = {0, 1,2}

Γ = {0, 1, 2, X, Y, Z, B}

q_0 = q_0

B = B

F = $\{q_6\}$



Transition Diagram

δ Transition table

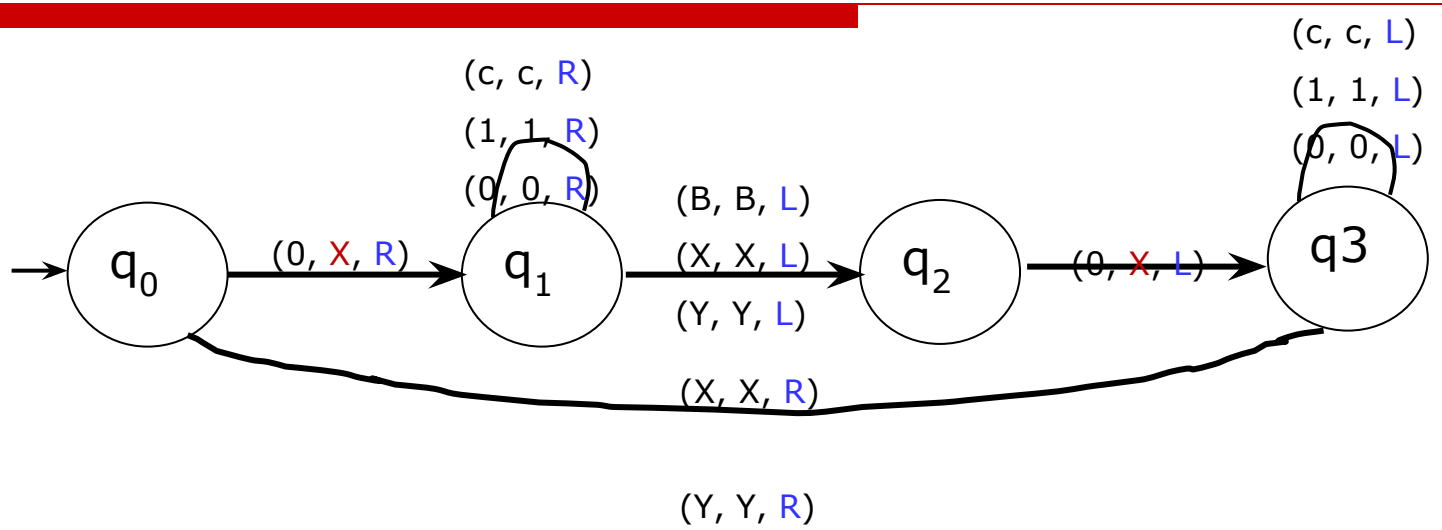
State	Input tape Symbols						
	0	1	2	X	Y	Z	B
q_0	(q_1, X , R)	---		---	(q4, Y , R)	--	--
q_1	(q_1, 0 , R)	(q_2, Y, R)		---	(q_1, Y , R)	---	---
q_2	---	(q_2, 1, R)	(q_3, Z, L)	---	---	(q_2, Z,R)	---
q_3	(q_3, 0 , L)	(q_3, 1, L)	--	(q_0,X,R)	(q_3, Y,L)	(q_3, Z,L)	---
q_4	---	---	---	---	(q_4, Y , R)	(q_5, Z , R)	---
q_5	---	---	---	---	---	(q_5, Z , R)	(q_6,B,R)
q_6	---	---	---	---	---	---	---

Question

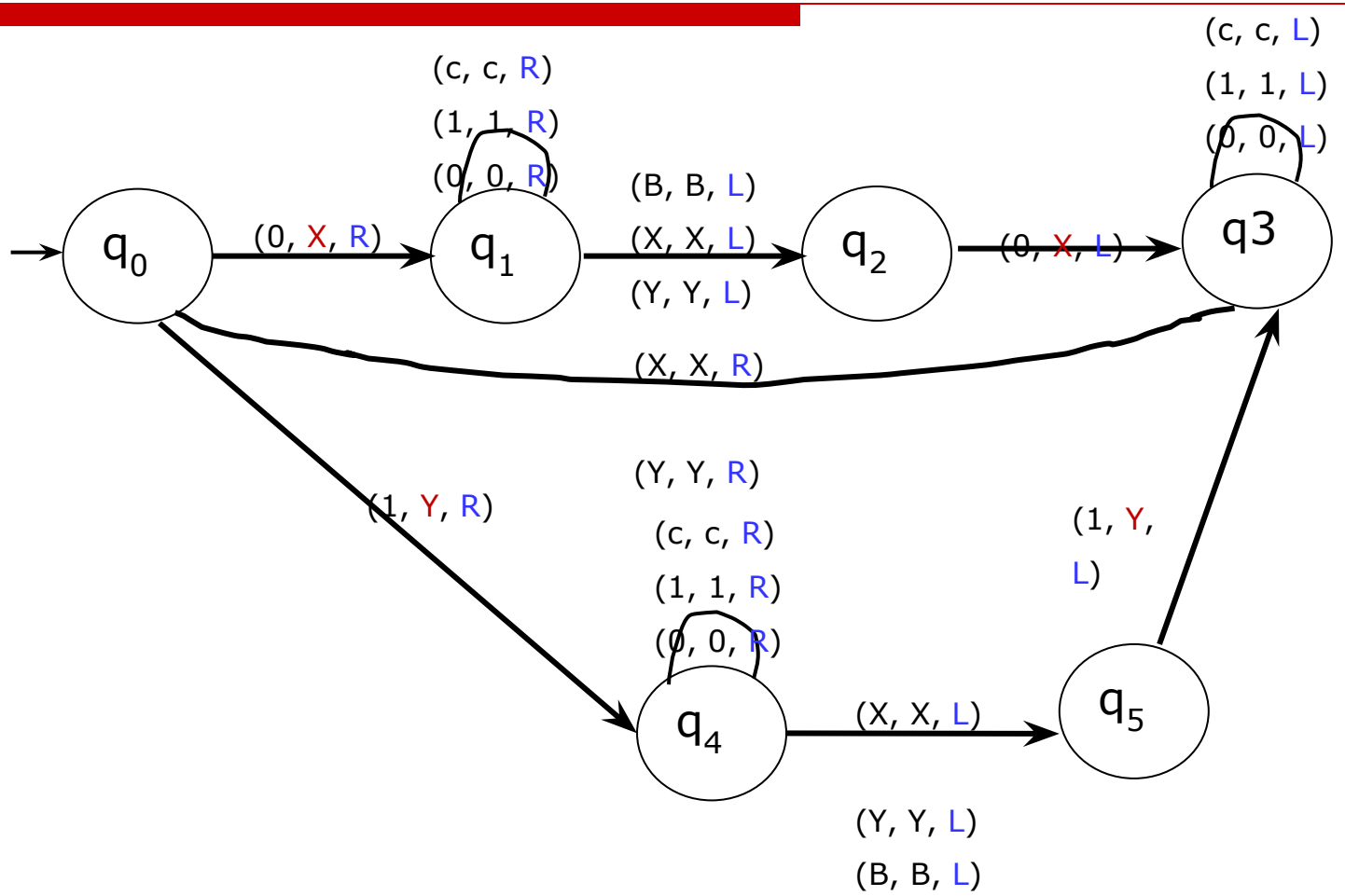
Design TM for $L=\{wcw^r, w \in (0+1)^*\}$

Valid string: 110c011

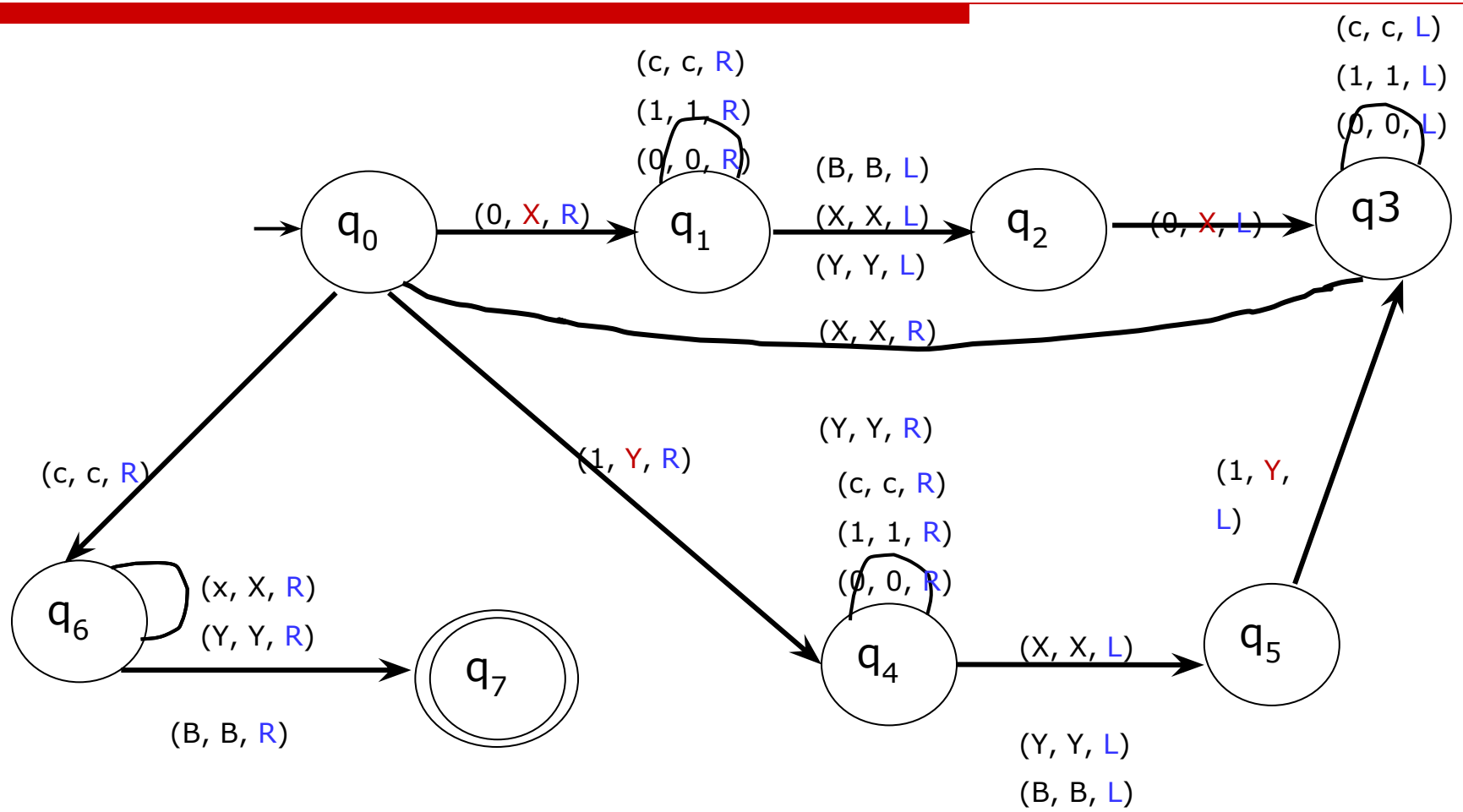
TM for L={wcw^r, w ∈ (0+1)*}



TM for L={wcw^r, w ∈ (0+1)*}



Complete TM for L={wcw^r, w ∈ (0+1)*}

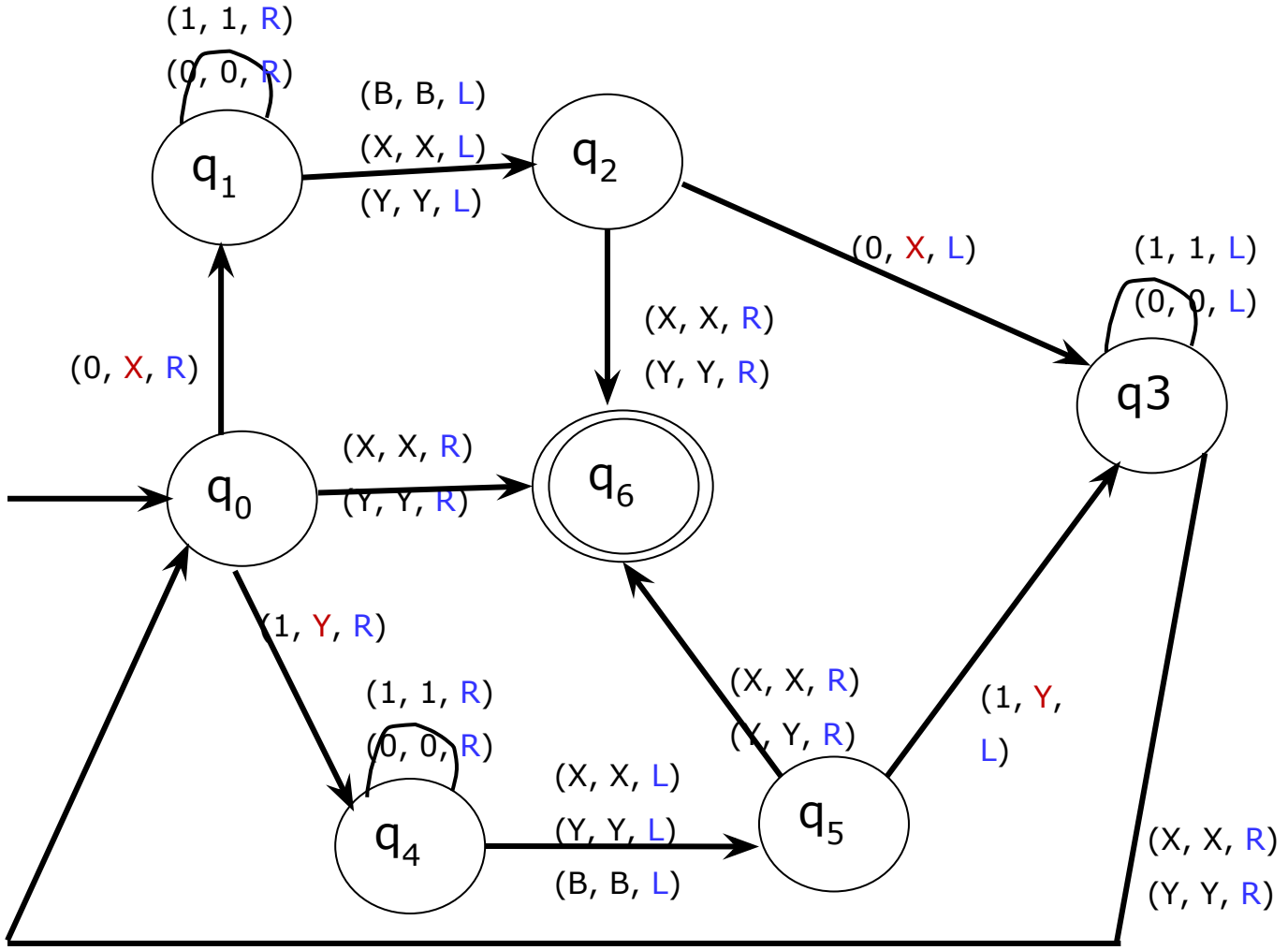


Question

Design TM which accepts the set of all palindromes over {0, 1}
Example:
Valid Strings:
010
0110

Invalid Strings:
1010
110

Complete design of TM which accepts the set of all palindromes over {0, 1}



Question

Design Turing Machine to compute the function, which is called monus or proper subtraction and is defined by

(m-n) = max (m-n, 0)

Note:

The monus operation is defined as

(m-n) = (m-n) if m **≥** n

(m-n) = 0 if m **<** n

Example:

If m = 5 , n =2 **then** (m-n) = (5-2) = **3**

If m = 2 , n =5 **then** (m-n) = (2-5) = **0**

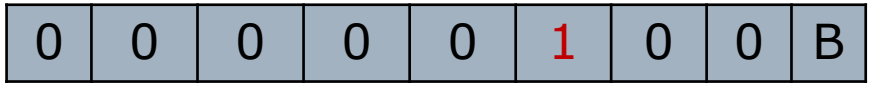
Design TM to compute the function, which is called monus or proper subtraction and is defined by (m-n) = max (m-n, 0)

To start with the tape consists of 0^m10^n which is terminated by blank symbol.

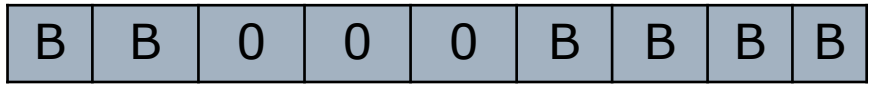
Example:

m=5 , n=2

Input tape



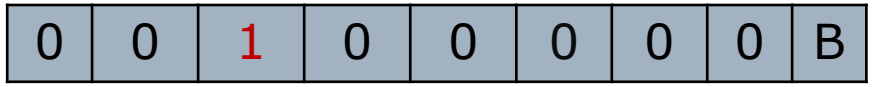
Output Tape



(5-2)=3

m=2 , n=5

Input tape



Output Tape

(2-5)=0

Design TM to compute the function, which is called monus or proper subtraction and is defined by
(m-n) = max (m-n, 0)

Rough Slide

To start with the tape consists of 0^m10^n which is terminated by blank symbol.

Example:

m=2 , n=5

Input tape

0	0	1	0	0	0	0	0	B

Output
Tape

B	B	B	B	B	B	B	B	B

(2-5)=0

Design TM to compute the function, which is called monus or proper subtraction and is defined by
(m-n) = max (m-n, 0)

Rough Slide

To start with the tape consists of 0^m10^n which is terminated by blank symbol.

Example:

m=5 , n=2

Input tape

0	0	0	0	0	1	0	0	B

Output Tape

B	B	0	0	0	B	B	B	B

(5-2)=3

Design TM to compute the function, which is called monus or proper subtraction and is defined by (m-n) = max (m-n, 0)

To start with the tape consists of 0^m10^n which is terminated by blank symbol.

Example: m=5 , n=2

0	0	0	0	0	1	0	0	B

General Approach: The sequence of 0's is partitioned into first group with **m** number of 0's followed by a 1 and followed by second group with **n** number of 0's. The machine finds leftmost 0 and is replaced by blank B. then move towards right to search for 1. After finding 1, it search's leftmost 0 in the second group and is replaced by 1 and move towards left to get leftmost 0 in the first group. The procedure is repeated till one of the following conditions are satisfied:

- When searching for a 0 in second group, if B is encountered it means that **n** number of 0's in the second group are replaced by 1's and (n+1) in the first group are changed to B's. Now, the second group will have (n+1) ones. The machine replaces (n+1) 1's by one 0 and **n** B's and observe that only (m-n) 0's exists on the tape.
- In the first group, if the machine can not find a 0 (Since first m 0's have already been changed to B's) it mean that (m<n) and so no 0's and 1's should be there on the tape.

Design TM to compute the function, which is called monus or proper subtraction and is defined by (m-n) = max (m-n, 0)

Input tape

0	0	0	0	0	1	0	0	B

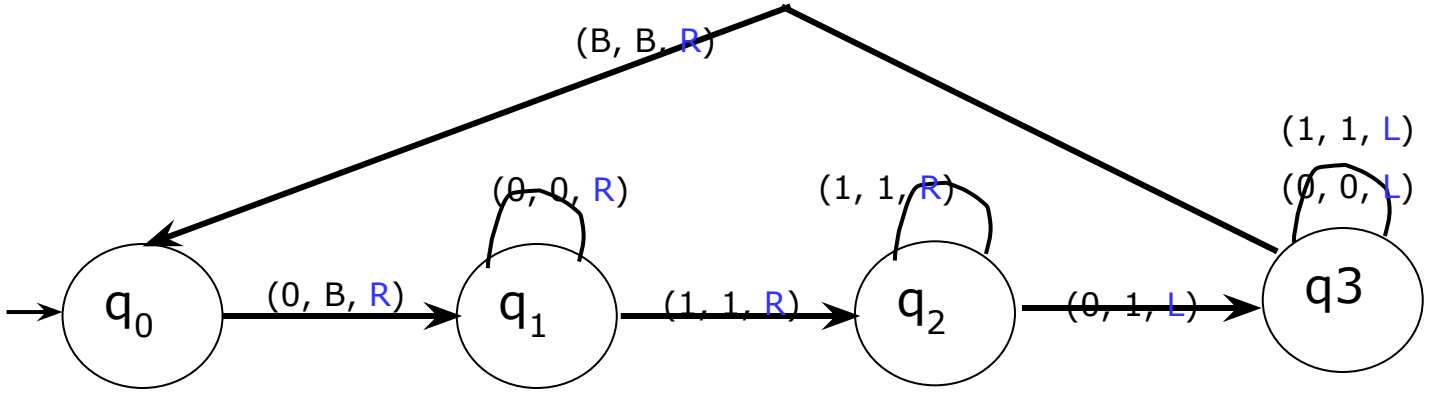
Output Tape (5-2)=3

B	B	B	0	0	0	B	B	B

Logic

0	0	0	0	0	1	0	0	B

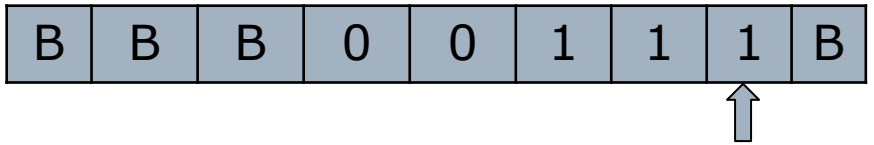
Step 1:
In state q_0, if 0 is encountered
- Replace 0 by B, Change state to q_1, Move right.
- Replace 0 by 0, be in same state q_1, Move right. When 1 is encountered, Change State to q_2, Move Right.
- Replace 1 by 1, be in same state q_2. When leftmost 0 is encountered replace by 1, change state to q_3, move left. When B is encountered replace by B, Move left, change state to q4, go to step 2
-Replace 0 by 0, 1 by 1, be same state q_3 . When B is encountered, replace by B, move right, go to state q_0. Repeat step 1
In state q_0, if 1 is encountered, go to step 3



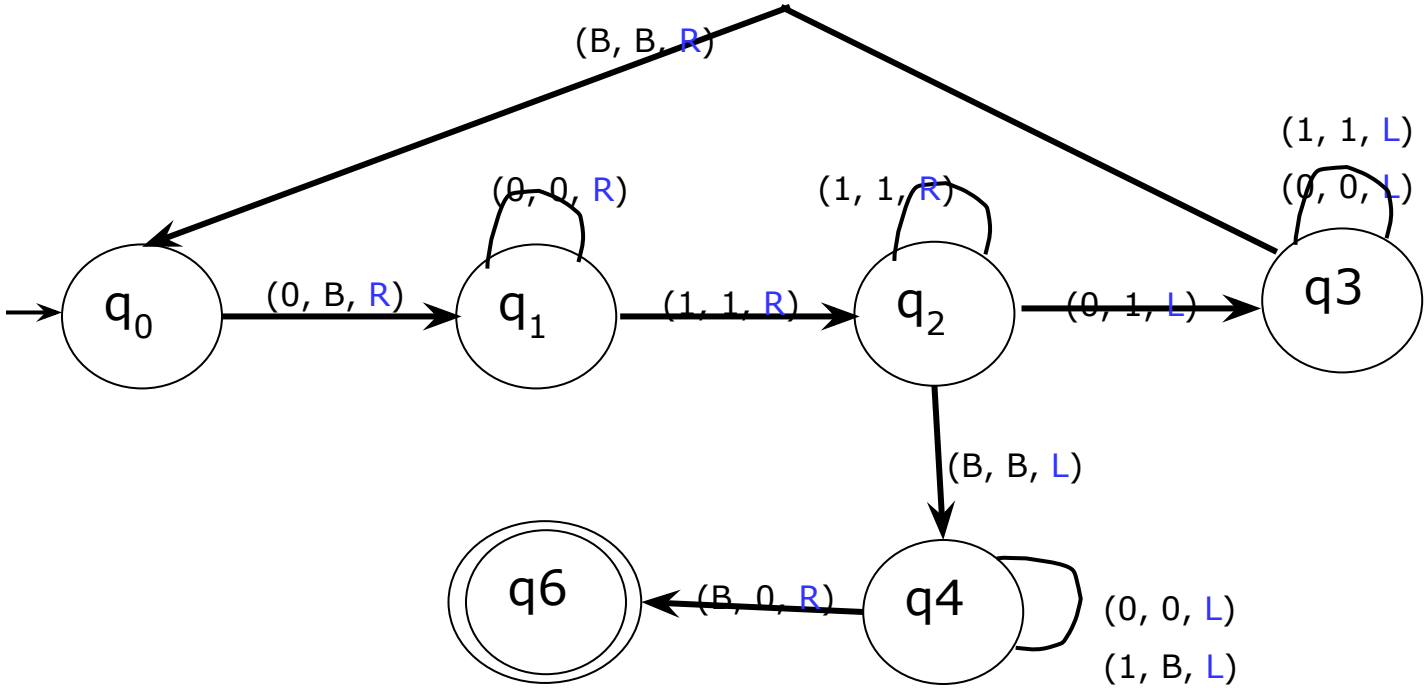
Design TM to compute the function, which is called monus or proper subtraction and is defined by (m-n) = max (m-n, 0)

Example for (5-2)=3

Logic



Step 2:
- Replace 0 by 0, Replace 1 by B, be in same state q4, Move left.
- If B is encountered replace by 0, move right, go to final state q_6



Design TM to compute the function, which is called monus or proper subtraction and is defined by (m-n) = max (m-n, 0)

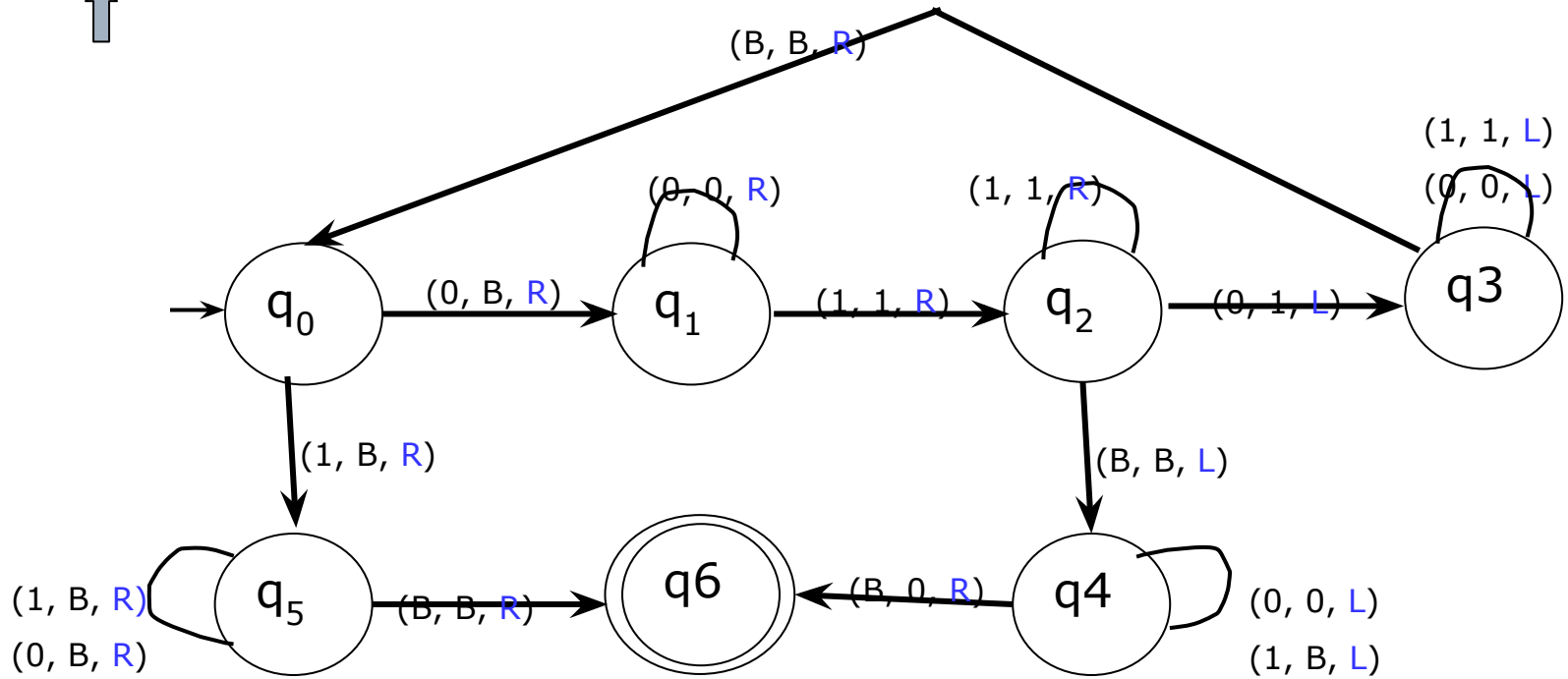
Input

Example for (2-5)=0

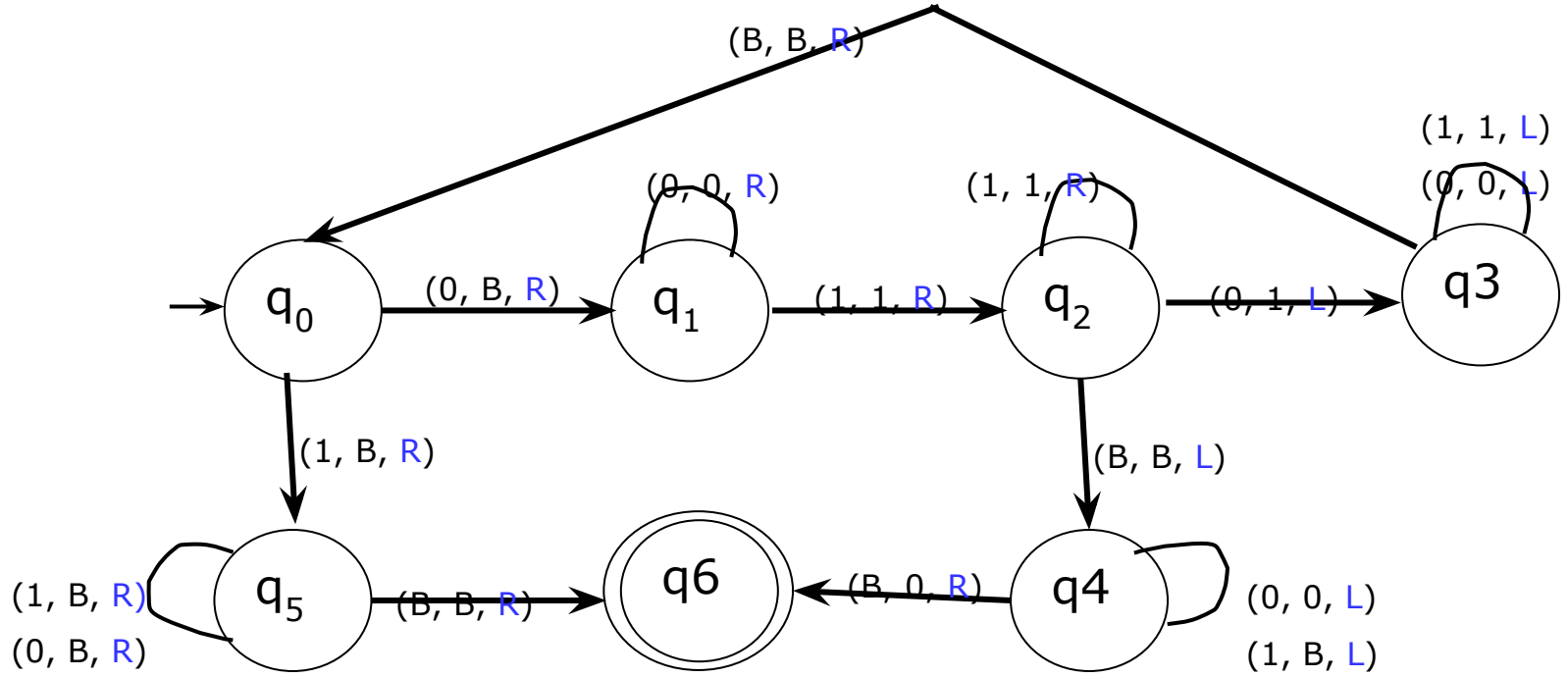
0	0	1	0	0	0	0	0	B

B	B	1	1	1	0	0	0	B

Step 3:
- Replace 1 by B, Replace 0 by B, be in same state q5, Move right.
- If B is encountered replace by B, move e right, go to final state q_6



Complete design of TM to compute the function, which is called monus or proper subtraction and is defined by (m-n) = max (m-n, 0)



Multiplication in TM (using Subroutines)

Example:

2x3 = 6

Input Tape:

1	1	0	1	1	1	0	B	B	B	B	B	B	B

Output Tape:

B	B	B	B	B	B	B	1	1	1	1	1	1	B

Multiplication in TM (using Subroutines)
Rough Slide

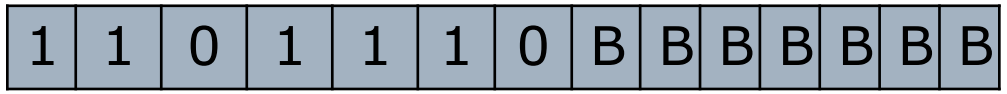
Example: 2x3 = 6

Input Tape:

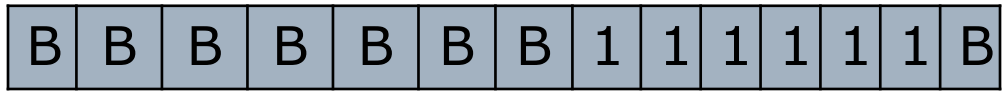
1	1	0	1	1	1	0	B	B	B	B	B	B	B

Multiplication in TM (using Subroutines)

Input Tape



Output Tape



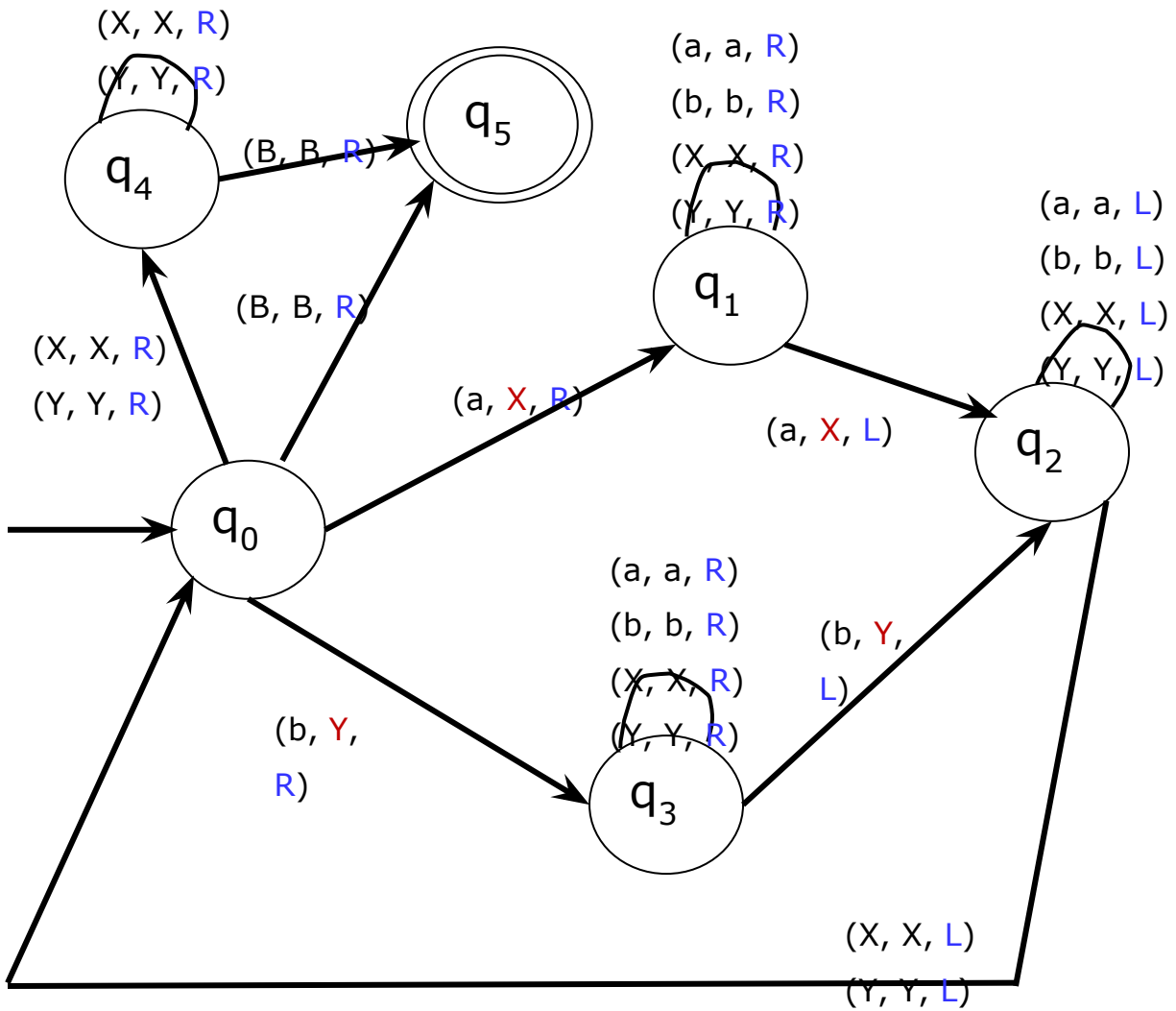
Question

Design TM for L={SS, S ∈(a+b)*)
Example:
Valid Strings:

aa
abab
aabaab

Invalid Strings:
ab
aaa
abba

Complete TM for L={SS, S ∈(a+b)*)

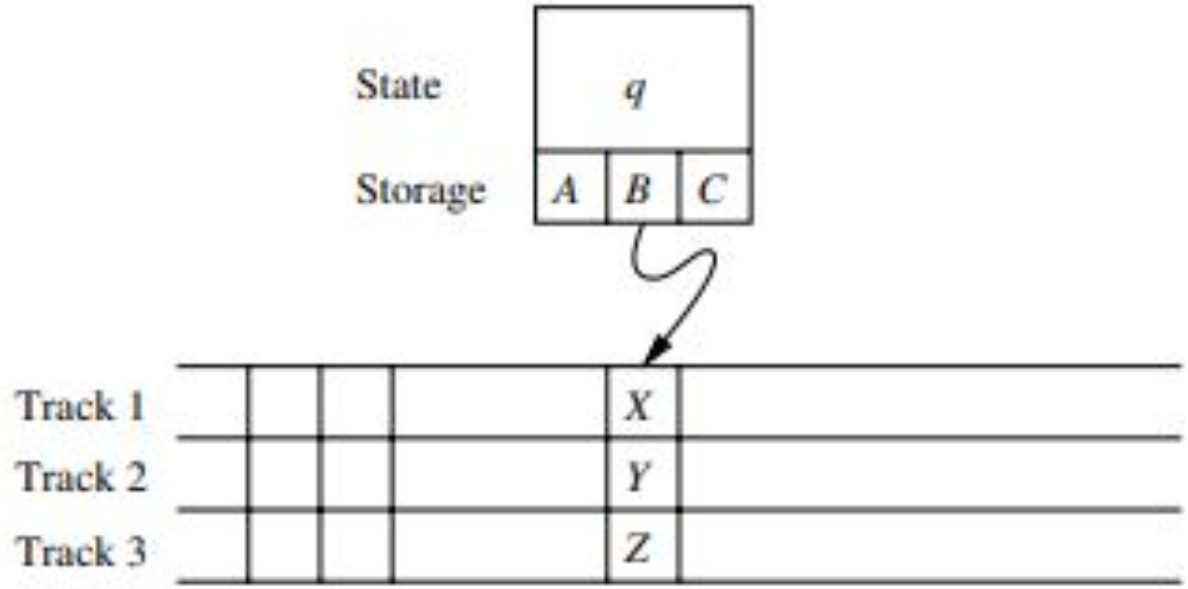


Programming Techniques for Turing Machine (TM)

- Storage in the state
- Multipule Tracks
- Subroutines

Storage in the State

We can use the finite control not only to represent a position in the program of the Turing machine but to hold a finite amount of data Figure



Mutiple Tracks TM and Subroutine TM

Mutiple Tracks TM:

Another useful trick is to think of the tape of a Turing machine as composed of several tracks. Each track can hold one symbol and the tape alphabet of the TM consists of tuples with one component for each.

Subroutine TM:

As with programs in general it helps to think of Turing machines as built from a collection of interacting components or subroutines A Turing machine subroutine is a set of states that perform some useful process. This set of states includes a start state and another state that temporarily has no moves and that serves as the return state to pass control to whatever other set of states called the subroutine. The call of a subroutine occurs whenever there is a transition to its initial state. Since the TM has no mechanism for remembering a return address that is a state to go to after it finishes, should our design of a TM call for one subroutine to be called from several states we can make copies of the subroutine using a new set of states for each copy. The calls are made to the start states of different copies of the subroutine and each copy returns to a different states.

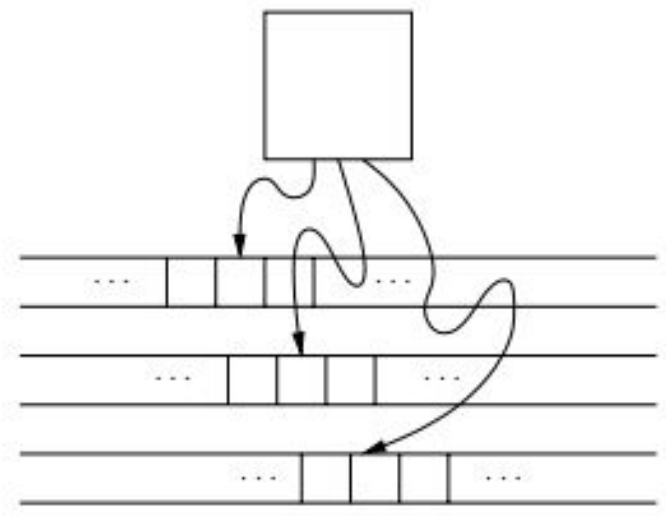
Extension of Basic Turing Machine (TM)

- Multitape TM

Mutitape TM

A multitape TM is as suggested by Fig below. The device has a finite control state and some finite number of tapes Each tape is divided into cells and each cell can hold any symbol of the finite tape alphabet As in the single tape TM the set of tape symbols includes a blank and has a subset called the input symbols of which the blank is not a member. The set of states includes an initial state and some accepting states Initially

1. The input a finite sequence of input symbols is placed on the first tape
2. All other cells of all the tapes hold the blank
3. The finite control is in the initial state
4. The head of the first tape is at the left end of the input
5. All other tape heads are at some arbitrary cell Since tapes other than the first tape are completely blank it does not matter where the head is placed initially all cells of these tapes look the same



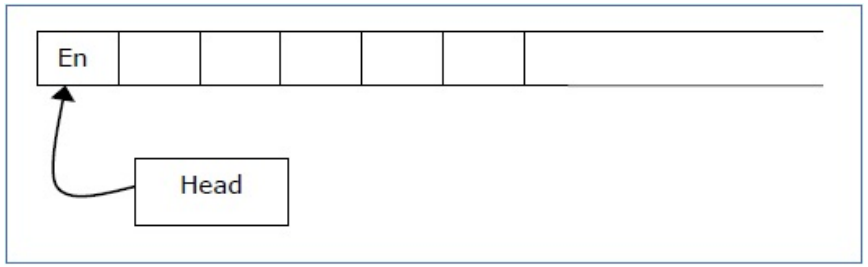
Multitape TUM

Restricted Turing Machine (TM)

- Semi-infinite tape TM
- Multi-stack TM

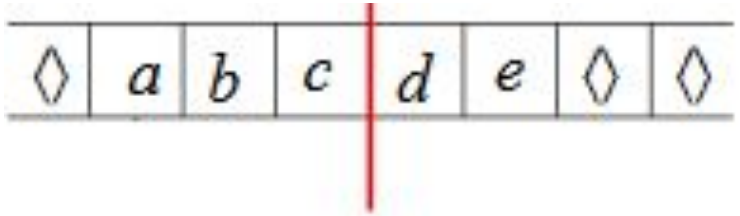
Semi-infinite Tape machine

A Turing Machine with a semi-infinite tape has a left end but no right end. The left end is limited with an end marker.



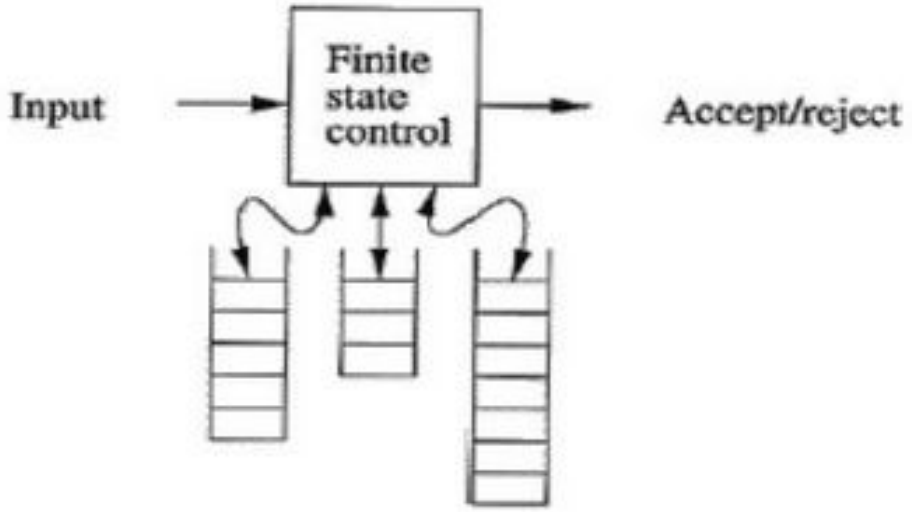
It is a two-track tape –

- **Upper track** – It represents the cells to the right of the initial head position.
- **Lower track** – It represents the cells to the left of the initial head position in reverse order.



Multi-stack Turing Machine

- Generalizations of the PDAs
- TMs can accept languages that are not accepted by any PDA with one stack.
- But PDA with two stacks can accept any language that a TM can accept.



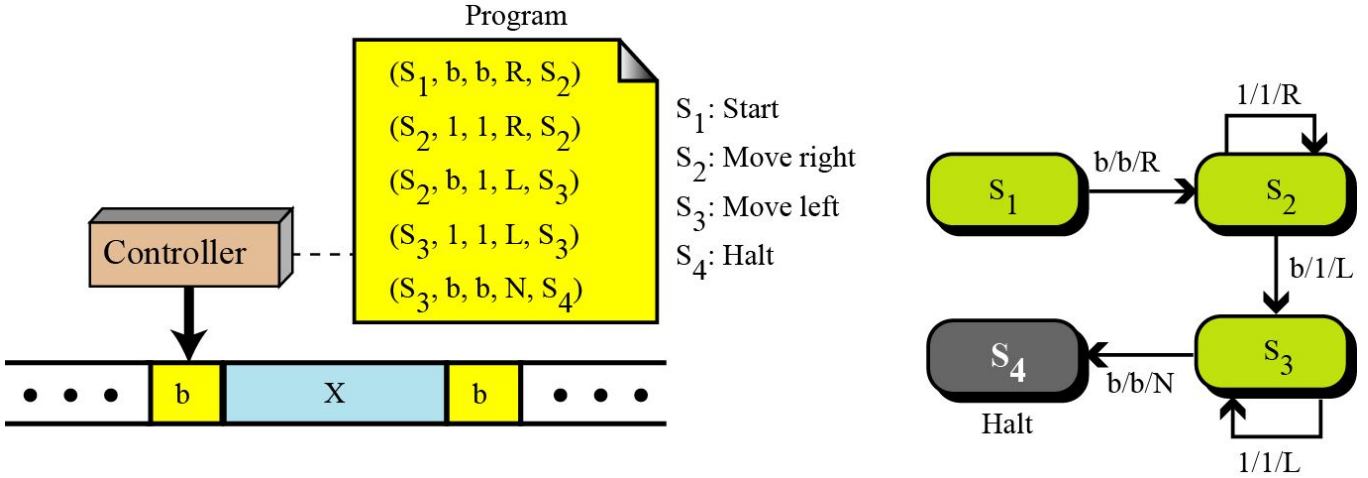
Multi-stack Turing Machine

- A k-stack machine is a deterministic PDA with k stacks.
- It obtains its input from an input source rather than having the input placed in a tape.
- It has a finite control, which is in one of a finite set of states.
- It has a finite stack alphabet, which it uses for all its stacks.
- A move of a multistack machine is based on:
  - The state of the finite control
  - The input symbol read, which is chosen from the finite input alphabet
  - The top stack symbol on each stack

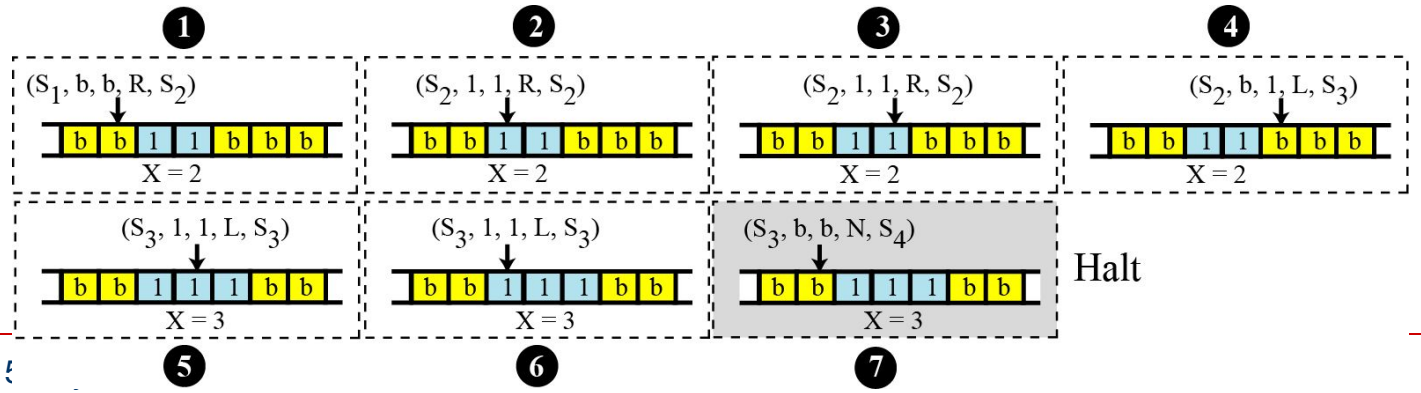
Turing Machine and Computers

Simulating computer by Turing machine

- Example (1): Increment Operation[incr(x)]

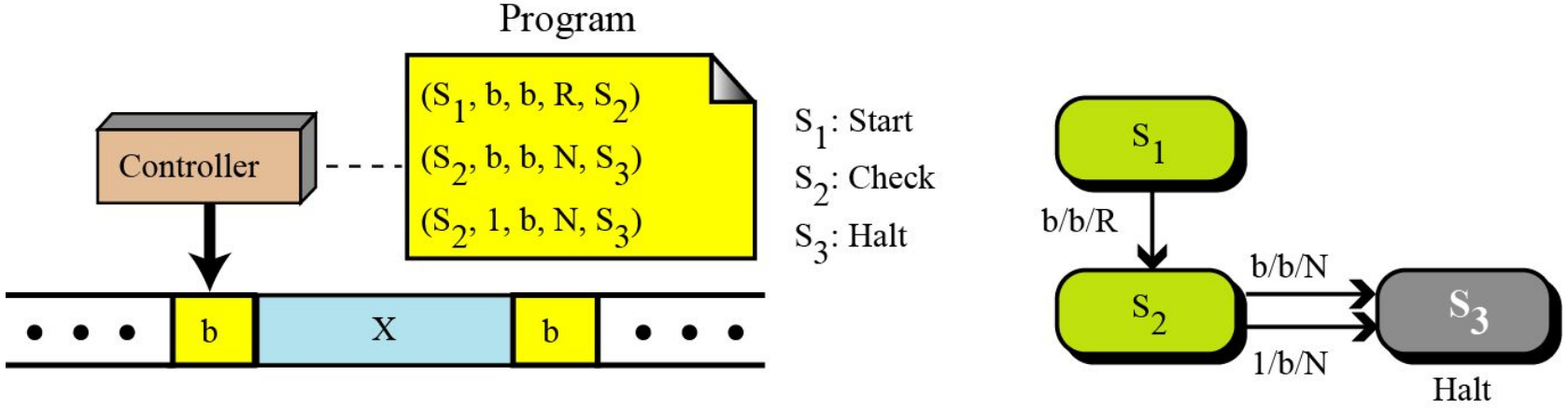


- Step by step operation which shows how the TM can increment x when x=2

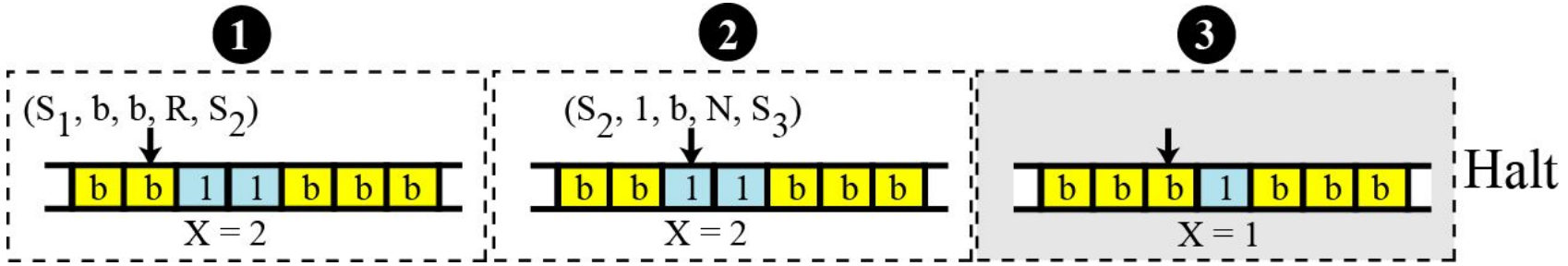


Simulating computer by Turing machine

- Example (2): Decrement Operation[decr(x)]



- Step by step operation which shows how the TM can decrement x when x=2



Introduction to Undecidable Problems

- Post Correspondence problem

Recursive and Recursively Enumerable language

Recursive Language:

- A language 'L' is said to be recursive if there exists a Turing machine which will accept all the strings in 'L' and reject all the strings not in 'L'.
- The Turing machine will halt every time and give an answer (accepted or rejected) for each and every string input.

Recursively Enumerable Language:

- A language 'L' is said to be a recursively enumerable language if there exists a Turing machine which will accept (and therefore halt) for all the input strings which are in 'L'.
- But may or may not halt for all input strings which are not in 'L'.

Decidable, Partially-Decidable and Undecidable language

Decidable Language:

A language 'L' is decidable if it is a recursive language. All decidable languages are recursive languages and vice-versa.

Partially Decidable Language:

A language 'L' is partially decidable if 'L' is a recursively enumerable language.

Undecidable Language:

- A language is undecidable if it is not decidable.
- An undecidable language may sometimes be partially decidable but not decidable.
- If a language is not even partially decidable, then there exists no Turing machine for that language

Undecidable Problems

Recursive Language	TM will always Halt
Recursively Enumerable Language	TM will halt sometimes & may not halt sometimes
Decidable Language	Recursive Language
Partially Decidable Language	Recursively Enumerable Language
UNDECIDABLE	No TM for that language

Undecidable problems

- Some problems take a very long time to solve, so we use algorithms that give approximate solutions. There are some problems that a computer can *never* solve, even the world's most powerful computer with infinite time.
- An **undecidable problem** is one that should give a "yes" or "no" answer, but yet no algorithm exists that can answer correctly on all inputs.

Halting Problem

Alan Turing proved the existence of undecidable problems in 1936 by finding an example,
the now famous "halting problem":
Based on its code and an input, will a particular program ever finish running?
For example, consider this program that counts down:

```
num ← 10
REPEAT UNTIL (num = 0) {
        DISPLAY(num)
        num ← num – 1
 }
```

That program will halt, since num eventually becomes 0.
Compare that to this program that counts up:

```
num ← 1
REPEAT UNTIL (num = 0) {
        DISPLAY(num)
        num ← num + 1
}
```

It counts up *forever*, since num will never equal 0.

Algorithms *do* exist that can correctly predict that the first program halts and the second program never does.
These are simple programs which don't change based on different inputs.

However, no algorithm exists that can analyze *any* program's code and determine whether it halts or not.

Post Correspondence Problem

- The Post Correspondence Problem (PCP), introduced by Emil Post in 1946, is an undecidable decision problem. The PCP problem over an alphabet ∑ is stated as follows –
- Given the following two lists, **M** and **N** of non-empty strings over ∑ –

  $M = (x_1, x_2, x_3, \ldots\ldots\ldots, x_n)$

  $N = (y_1, y_2, y_3, \ldots\ldots\ldots, y_n)$
- We can say that there is a Post Correspondence Solution, if for some $i_1, i_2, \ldots\ldots\ldots\ldots i_k$, where $1 \leq i_j \leq n$, the condition $x_{i1} \ldots\ldots x_{ik} = y_{i1} \ldots\ldots y_{ik}$ satisfies.

Post Correspondence Problem

Example 1: Find whether the lists
M = (abb, aa, aaa) and
N = (bba, aaa, aa) have a Post Correspondence Solution?

Solution

	x_1	x_2	x_3
M	Abb	aa	aaa
N	Bba	aaa	aa

Here, $x_2x_1x_3$ = **'aaabbaaa'** and $y_2y_1y_3$ = **'aaabbaaa'**
We can see that $x_2x_1x_3 = y_2y_1y_3$
Hence, the solution is **i = 2, j = 1, and k = 3**

Post Correspondence Problem

Example 2: Find whether the lists **M = (ab, bab, bbaaa)** and **N = (a, ba, bab)** have a Post Correspondence Solution ?

Solution:

	x_1	x_2	x_3
M	ab	bab	bbaaa
N	a	ba	bab

In this case, there is no solution because –

$| x_2x_1x_3 | \neq | y_2y_1y_3 |$ (Lengths are not same)

Post Correspondence Problem

Example

	A	B
i	w_i	x_i
1	1	111
2	10111	10
3	10	0

This PCP instance has a solution: 2, 1, 1, 3:

$$w_2w_1w_1w_3 = x_2x_1x_1x_3 = 101111110$$

Post Correspondence Problem (PCP)

Does this PCP instance have a solution?

	A	B
i	w_i	x_i
1	110	110110
2	0011	00
3	0110	110

This PCP instance has a solution: 2,3,1

$w_2w_3w_1 = x_2x_3x_1 = 00110110110$

One more solution:

2,1,1,3,2,1,1,3

Thanks for Listening

Program Halting Problem

- **Input:** a program P in some programming language
- **Output: true** if P terminates; **false** if P runs forever.

Examples

halts("2+2")

True

halts("**def f(n):**
if n==0: return 1
else: return n * f(n-1)
f(10)")

halts("**def f(n):**
if n==0: return 1
else: return n * f(n-1)
f(10.5)")

True

False

Tougher Example

```
halts("
    def isPerfectNumber(n): # n is perfect if
factors sum to n
    divs = findDivisors(n)
    return n == sum(divs)
     i = 3
     while not isPerfectNumber (i): i = i + 2
     print i")
```

Unknown

Note: it is unknown where an odd perfect number exists. (Numbers up to 10^{300} have been tried without finding one yet.)